

## FINAL DESIGN DOCUMENT

### ROBO-GOOSE

Amie J Sallah [CSE / MTH, UB | [amiesall@buffalo.edu](mailto:amiesall@buffalo.edu)]

Nicholas G Medina [CSE, UB | [ngmedina@buffalo.edu](mailto:ngmedina@buffalo.edu) || [nickmedina5991@gmail.com](mailto:nickmedina5991@gmail.com)]

Annie K Slenker [CSE, UB | [annaslen@buffalo.edu](mailto:annaslen@buffalo.edu)]

Samuel W Tiso [CSE, UB | [swtiso@buffalo.edu](mailto:swtiso@buffalo.edu)]

Austin J Smith [CSE, UB | [smith43@buffalo.edu](mailto:smith43@buffalo.edu)]

Thien Phuc Le [CSE, UB | [ptle@buffalo.edu](mailto:ptle@buffalo.edu)]

Ahmed Chowdhury [CSE, UB | [auchowdh@buffalo.edu](mailto:auchowdh@buffalo.edu)]

Eric Butcher [CSE, UB | [ericbutc@buffalo.edu](mailto:ericbutc@buffalo.edu)]

Samuel Magaseyev [CSE, UB | [ssmagase@buffalo.edu](mailto:ssmagase@buffalo.edu) ]

David Doermann [ CSE Department Chair | [doermann@buffalo.edu](mailto:doermann@buffalo.edu) ]

Kris Schindler [ Instructor | [kds@buffalo.edu](mailto:kds@buffalo.edu) ]

**Github Org:** <https://github.com/cse-450-ROBO-GOOSE>

# TABLE OF CONTENTS

## Contents

|                                            |    |
|--------------------------------------------|----|
| 1. EXECUTIVE SUMMARY .....                 | 6  |
| 1.1 PROJECT OVERVIEW .....                 | 6  |
| 1.2 PURPOSE OF THIS DOCUMENT .....         | 6  |
| 1.3 CONCEPT SUMMARY .....                  | 6  |
| 2. SYSTEM REQUIREMENTS .....               | 7  |
| 2.1 FUNCTIONAL REQUIREMENTS.....           | 7  |
| Locomotion & Movement.....                 | 7  |
| Control and Stability.....                 | 7  |
| Chassis and Structure .....                | 7  |
| Peripherals and Interaction.....           | 7  |
| Power and Electronics.....                 | 7  |
| 2.2 NON-FUNCTIONAL REQUIREMENTS .....      | 8  |
| Performance Constraints .....              | 8  |
| Mechanical Constraints .....               | 8  |
| Electrical and Safety Constraints .....    | 8  |
| Software and Reliability Constraints ..... | 8  |
| 3. SYSTEM OVERVIEW .....                   | 9  |
| 3.1 HARDWARE ARCHITECTURE .....            | 9  |
| 3.2 SOFTWARE ARCHITCTURE .....             | 10 |
| Tick Rates in the Main Loop.....           | 10 |
| Inter-board communication:.....            | 10 |
| State Machine.....                         | 11 |
| 3.3 SYSTEM ARCHITECTURE .....              | 11 |
| 3.3.1 SPI Bus (CAN Controller).....        | 12 |
| 3.3.2 I2C Bus.....                         | 12 |
| 3.3.3 Serial Buses .....                   | 13 |
| 3.3.4 CAN Bus Topology .....               | 13 |

|                                           |    |
|-------------------------------------------|----|
| 3.4 HARDWARE PINOUT .....                 | 14 |
| 4. DESIGN & IMPLEMENTATION .....          | 15 |
| 4.1 LOCOMOTION SYSTEM .....               | 15 |
| Wheel Motors (Main Movement) .....        | 15 |
| Stability and Control.....                | 15 |
| Legs .....                                | 15 |
| 4.1.1 Block Diagram .....                 | 16 |
| 4.1.2 Motor Diagrams .....                | 17 |
| 4.1.3 Motor Flowcharts .....              | 18 |
| 4.2 ELECTRICAL DESIGN.....                | 19 |
| Power Distribution .....                  | 19 |
| Fritzing Diagram .....                    | 20 |
| 4.3 BODY STRUCTURE .....                  | 21 |
| Global Mass Constraint .....              | 21 |
| Body – Outer Shell.....                   | 21 |
| Body – Chassis .....                      | 22 |
| Legs .....                                | 22 |
| Materials .....                           | 24 |
| Center of Mass (CoM).....                 | 24 |
| Stability Limit.....                      | 24 |
| Mobility Goals.....                       | 24 |
| Leg Joint Configuration .....             | 25 |
| 4.4 PERIPHERALS .....                     | 26 |
| Controller .....                          | 26 |
| Bluepad32 and Technical Information ..... | 26 |
| Candy Hopper .....                        | 30 |
| DFPlayer Mini Audio + Amp / Speaker ..... | 31 |
| 16x2 I2C LCD .....                        | 32 |
| BNO055 IMU .....                          | 33 |

|                                                     |    |
|-----------------------------------------------------|----|
| Built-in Web Server.....                            | 36 |
| RGB LED Eyes .....                                  | 37 |
| Spinning Hat .....                                  | 37 |
| 5. ASSEMBLY .....                                   | 38 |
| 6. BUILD & FLASH .....                              | 41 |
| 6.1 REQUIRED SOFTWARE .....                         | 41 |
| 6.2 SETTING UP PlatformIO IN VSCODE .....           | 41 |
| 6.3 PlatformIO CONFIGURATION.....                   | 42 |
| 6.4 BUILDING & FLASHING .....                       | 42 |
| 6.5 DEBUGGING VIA SERIAL.....                       | 43 |
| 6.6 SETTING UP Bluepad32 .....                      | 43 |
| 6.6.1 Install the Bluepad32 board package .....     | 43 |
| 6.6.2 Configure board settings in Arduino IDE ..... | 43 |
| 6.6.3 Upload the code (Nano bootloader mode) .....  | 44 |
| 6.6.4 Pair the controller.....                      | 44 |
| 7. QUICK START GUIDE .....                          | 45 |
| 8. TUNING GUIDE .....                               | 46 |
| 9. DEBUG GUIDE .....                                | 48 |
| 10. KNOWN ISSUES / FUTURE WORK .....                | 50 |
| 11. REFERENCES .....                                | 51 |
| 12. PROJECT HISTORY HIGHLIGHTS.....                 | 52 |
| Design Philosophy .....                             | 52 |
| Procurement .....                                   | 52 |
| Hardware .....                                      | 53 |
| Software Architecture Decisions .....               | 53 |
| LiPo Safety .....                                   | 53 |
| Workflow Lessons .....                              | 54 |
| Key People.....                                     | 54 |
| 13. APPENDIX .....                                  | 55 |

|                                  |    |
|----------------------------------|----|
| Body Equation .....              | 55 |
| Approximating battery power..... | 56 |

# 1. EXECUTIVE SUMMARY

## 1.1 PROJECT OVERVIEW

ROBO-GOOSE is the inaugural project for the CSE Showcase initiative. Its main purpose is to highlight the strengths of the Computer Engineering major and distinguish it from Computer Science within the University at Buffalo's CSE Department. The robot will be showcased during departmental events such as the CSE Open House and Accepted Students Day, where the Computer Engineering sub-department can use it as a conversation piece to demonstrate the hardware/software integration approach and defines and differentiates Computer Engineering from other majors.

## 1.2 PURPOSE OF THIS DOCUMENT

This design document consolidates all engineering work completed by the MOTOR, BODY-LEGS, and PERIPHERALS sub-teams.

It serves three primary roles:

- Blueprint for construction: A reader should be able to reproduce ROBO-GOOSE from this document alone
- Design justification record: Each subsystem includes the reasoning, calculations, and research that support design decisions
- Continuity for future teams: Teams building new iterations of ROBO-GOOSE can reference this work, evaluate assumptions, and introduce improvements based on documented rationale

## 1.3 CONCEPT SUMMARY

ROBO-GOOSE is a wheeled robot styled after a goose. It integrates:

- A pair of motorized wheels that provide primary motion
- Two-segment mechanical legs with knee and hip joints to support balance (Hip and knee joints were originally to be integrated with motors for extensive movement, however motor parts were never received during the duration of the class)
- A rigid central chassis designed to house electronics, batteries, hopper design and mechanical components
- A detachable outer shell that provides the visual goose form and protects internal components
- Special features that give the robot personality and to make it interactive (i.e. audio playback, RGB LED eyes, anti-tip motion).

## 2. SYSTEM REQUIREMENTS

### 2.1 FUNCTIONAL REQUIREMENTS

These requirements define the behavior the system must exhibit for ROBO-GOOSE to operate as intended.

#### **Locomotion & Movement**

- Robot must move forward and backward using wheel motors
- Robot must turn left and right by changing wheel speeds
- The legs must be able to lock into a stiff standing mode

#### **Control and Stability**

- The robot must stay balanced while moving
- PID control must keep wheel speeds steady and correct small tilts
- The robot must handle small bumps or pushes without falling over

#### **Chassis and Structure**

- The chassis must hold all internal parts (battery, motors, boards, wiring)
- The chassis must support the full weight of the robot
- The center of mass must stay low and inside the chassis footprint to avoid tipping over

#### **Peripherals and Interaction**

- LED eyes must be able to show different colors to signify different moods
- The hopper must release one candy at a time using a servo
- The speaker must be able to play a honk sound, funny sounds, and songs
- The propeller hat motor must spin on command
- The robot must respond to a controller for movement and mode selection

#### **Power and Electronics**

- The batteries must power all motors and peripherals at the same time
- Power distribution must be safe and must protect components

## 2.2 NON-FUNCTIONAL REQUIREMENTS

These describe constraints the robot must follow regarding performance, safety, and usability.

### **Performance Constraints**

- The total robot weight must be 18 kg or less
- The battery must last at least one hour during demonstrations
- Wheel motors must provide enough torque for smooth acceleration

### **Mechanical Constraints**

- The chassis must be stiff and not bend during movement
- Leg joints must move smoothly without wobbling
- The outer shell must be easy to remove without disturbing internal components

### **Electrical and Safety Constraints**

- Most wiring must be secured and protected inside the chassis
- Motor drivers must have overcurrent protection

### **Software and Reliability Constraints**

- Control loops must run consistently and in real time
- The software must prevent unsafe, sudden, or extreme motor motions
- The user must be able to interrupt any peripheral action at any time
- The software will be able to gracefully handle errors and drops in communication

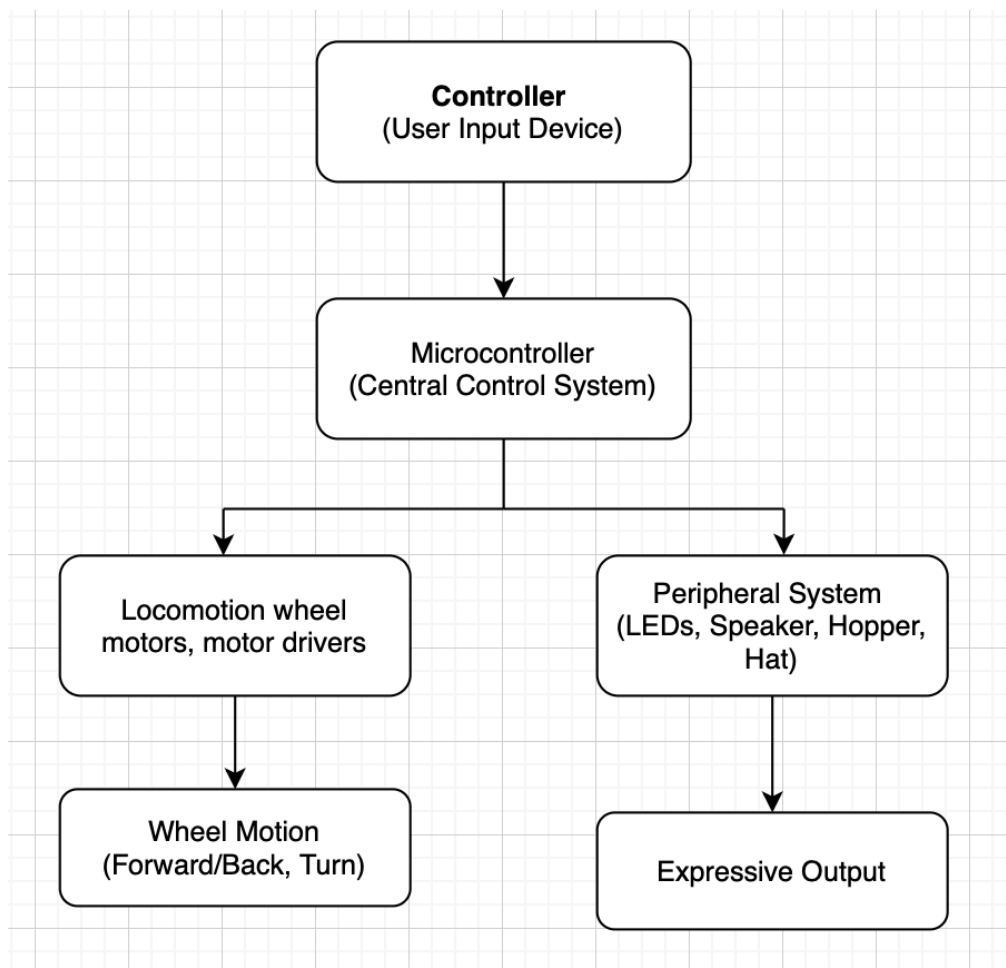


## 3. SYSTEM OVERVIEW

### 3.1 HARDWARE ARCHITECTURE

ROBO-GOOSE uses three microcontrollers rather than a single central one. The Arduino Mega 2560 is the main brain — it handles motor control, anti-tip protection, audio playback, the LCD, eye animations, and the hopper. Two Arduino Nano ESP32s handle jobs the Mega can't manage on its own: one bridges Bluetooth to the Xbox controller using the Bluepad32 library, and the other hosts a WiFi access point and the web soundboard interface.

The Mega controls the drive motors over a CAN bus through an MCP2515 SPI breakout. CAN runs at 1 Mbps to two REV Spark MAX motor controllers, which drive two REV NEO brushless motors through AndyMark sport gearboxes and HiGrip wheels. A third caster wheel keeps the robot upright as a baseline. A BNO055 IMU shares the I2C bus with a 16x2 LCD and provides real-time pitch data to the Mega. A DFPlayer Mini reads MP3s from an SD card and outputs through a MAX98306 amplifier to the speaker. RGB LED eyes and the hopper servo connect directly to PWM pins on the Mega.



## 3.2 SOFTWARE ARCHITCTURE

Code can be accessed at the following Github organization:

<https://github.com/cse-450-ROBO-GOOSE>

Any code needed for a hardware device such as the ESP32 nanos or Arduino Mega microcontroller can be cloned from the repositories in the above organization. Repositories are labelled to correspond with what they control or apply to. The code for the Mega lives in the Can-Control fork repository. Other repositories include one for sound output and interactable webserver, and one for Bluetooth Xbox controller remote control.

Files need to be flashed onto hardware using platform.io extension, which should be set up locally in an IDE prior to cloning the repositories or beginning to flash code to hardware. More detailed guides on flashing code to hardware can be found in Arduino documentation as well as section 6 of this document for whichever device is being utilized.

The software architecture on the main robot microcontroller was planned to utilize both of the ARM cores available on the Arduino Giga. The system will run the main motor control code on the more powerful M7 core of the Arduino Giga and will run the peripheral code which handles Bluetooth communication and controls the peripherals on the weaker M4 core. There will be shared global variables which the M4 core will write to with data from the Bluetooth connection and which both cores will read when performing actions.

The ROBO-GOOSE v1 had its firmware run on an Arduino Mega and two Arduino Nano ESP32s because the Giga did not arrive in time to complete the implementation of the firmware on the Giga. Future iterations of the ROBO-GOOSE should consider firmware functionality onto a single microcontroller using a single codebase.

The code is a single `main.cpp` file built with PlatformIO. There's one cooperative scheduler in `loop()` that runs each subsystem at its own interval. There are no blocks or interrupts used, since everything is polled, as well as no `delay()` function inside the main loop.

### Tick Rates in the Main Loop

- Motor ramp: every 1ms for a smooth feel
- CAN Heartbeat + Motor Commands: every 10ms
- IMU read + LED animation: every 20ms (50Hz)
- LCD refresh: every 500ms (or 150ms in IMU tuning mode)
- Telemetry to soundboard Nano: every 500ms
- Serial Debug Printout: every 250ms

**Inter-board communication:** The Mega uses three of its hardware serial ports. Serial1 receives one-way CSV input from the controller Nano at 9600 baud. Serial2 talks to the

DFPlayer Mini. Serial3 exchanges ASCII commands with the soundboard Nano. The soundboard Nano sends commands like `MUSIC:05` or `VOL:UP`, and the Mega sends `STATS:` telemetry lines back every 500ms.

## State Machine

- LED Breathing: 4-state blink animation (hold bright → fade dim → hold dim → fade bright)
- DJ mode: intro play, then transitions to music after 5s
- Three-stage anti-tip: brake softening → catch boost → death mode

## 3.3 SYSTEM ARCHITECTURE

The hardware is organized around the Mega as the central hub. Everything else either feeds data into the Mega or takes commands from it. The Mega firmware is a single `main.cpp` file built with PlatformIO. All logic runs through a `loop()` that polls each subsystem at its own interval.

### Input:

- Xbox controller pairs to the Nano ESP32, which sends CSV of inputs over Serial1 to the Mega for parsing
- Web soundboard runs on a second Nano ESP32 and sends commands over Serial3 to the Mega
- BNO055 IMU shares the I2C bus with the LCD: gives the Mega real-time pitch data

### Mega's Output:

- Two REV NEO motors via Spark MAX controllers, controlled over CAN bus through an MCP2515 SPI breakout
- DFPlayer Mini audio module over Serial2 plays MP3 files from an SD card
- 16x2 I2C LCD for live data and diagnostics
- RGB LED eyes via PWM
- Hooper Servo via PWM

### Power:

- 10000mAh 12V 3S LiPo battery feeds the Arduino Mega, Buck Converter, and Spark MAXes directly through a Wago power distribution hub.
- Buck Converter: Steps 12V down to 5V into the Rev Mini Power Module.
- The REV Mini Power Module distributes regulated 5V to MCP2515, DFPlayer Mini + Amplifier Board, Arduino Mega (GND), Hopper, Soundboard Arduino Nano (WiFi), 16x2 LCD Screen.
- Common ground ties across all subsystems.

### 3.3.1 SPI Bus (CAN Controller)

The MCP2515 CAN breakout sits on the Mega's hardware SPI pins (50/51/52) with chip select on 53. Library is autowp/autowp-mcp2515. The CAN bus runs at 1 Mbps to match Spark MAX expectations, with the breakout's 8 MHz crystal as the time reference.

The MCP2515's INT pin is wired to Mega pin 2 but the code never uses it. Instead, `drainCANStatus()` polls every loop iteration. This is simpler than interrupt-driven RX and runs fast enough that we don't miss frames.

#### Key CAN frame IDs (Spark MAX API):

- 0x2050080 - duty cycle command
- 0x2051840 - status frame 1 (velocity, temp, voltage)
- 0x2052C80 - heartbeat (keeps Spark MAXes out of safety lockout)

The voltage decoding from `status_1` frames is non-obvious: motor bus voltage is a 16-bit value spanning bytes 5 and 6 (low byte first). The code computes  $((\text{data}[6] \ll 8) | \text{data}[5]) * 0.00782f$  to calculate battery voltage. This calibration factor was reverse engineered by dumping raw bytes and watching them change as the battery drained.

The team was able to use the Spark MAXes through an MCP2515 and Arduino Mega without a roboRIO thanks to Will Guimont's CANControl GitHub repo, which provides high-level C++ wrappers to control FRC motor controllers without a roboRIO. See:

<https://github.com/willGuimont/CanControl>

### 3.3.2 I2C Bus

Two devices share the I2C bus on pins 20 (SDA) and 21 (SCL). The 16x2 LCD lives at address `0x27` (some LCD backpacks use `0x3F` instead). The BNO055 IMU sits at `0x28`. Both work together because they have different addresses.

Libraries: marcoschwartz/LiquidCrystal\_I2C for the screen, Adafruit BNO055 plus Adafruit Unified Sensor for the IMU.

### 3.3.3 Serial Buses

| Port    | Pins         | Purpose                          | Direction                     |
|---------|--------------|----------------------------------|-------------------------------|
| Serial0 | USB          | Debug console                    | Both                          |
| Serial1 | RX=19        | Controller input from Nano ESP32 | One-way (Nano → Mega)         |
| Serial2 | TX=16, RX=17 | DFPlayer Mini audio module       | Both (mostly Mega → DFPlayer) |
| Serial3 | TX=14, RX=15 | Soundboard Nano (web server)     | Both                          |

All ports run at 9600 baud except Serial0 which runs at 115200 for debug.

### 3.3.4 CAN Bus Topology

MCP2515 → Spark MAX 1 → Spark MAX 2, with a 120  $\Omega$  termination resistor at the end. The heartbeat frame is sent every 10 ms regardless of what else is happening. Without it, the Spark MAXes drop into a safety-disabled state after about 100 ms.

## 3.4 HARDWARE PINOUT

All assignments are at the top of `main.cpp` in the `//PIN ASSIGNMENTS` block. Pins 6 and 7 are used to drive a propeller hat motor and now free for future use.

| Mega Pin | Function      | Notes                          |
|----------|---------------|--------------------------------|
| 0        | Serial0 USB   | Debug console only             |
| 14       | Serial3 TX    | To soundboard Nano             |
| 15       | Serial3 RX    | From soundboard Nano           |
| 16       | Serial2 TX    | To DFPlayer Mini               |
| 17       | Serial2 RX    | From DFPlayer Mini             |
| 19       | Serial1 RX    | From Controller Nano (one-way) |
| 20       | I2C SDA       | Shared: 16x2 LCD + BNO055 IMU  |
| 21       | I2C SCL       | Shared: 16x2 LCD + BNO055 IMU  |
| 50       | SPI MISO      | MCP2512 CAN                    |
| 51       | SPI MOSI      | MCP2512 CAN                    |
| 52       | SPI SCK       | MCP2512 CAN                    |
| 53       | SPI CS        | MCP2512 CAN                    |
| 2        | CAN INT       | Wired but unused               |
| 3        | RGB LED Red   | PWM                            |
| 4        | RGB LED Green | PWM                            |
| 5        | RGB LED Blue  | PWM                            |
| 6        | DRV8871 IN2   | Propeller speed (unused)       |
| 7        | DRV8871 IN1   | Propeller direction (unused)   |
| 8        | Hopper Servo  | PWM                            |

## 4. DESIGN & IMPLEMENTATION

### 4.1 LOCOMOTION SYSTEM

ROBO-GOOSE uses one category of motors: wheel motors for primary movements. (*Motor calculations in appendix*)

#### Wheel Motors (Main Movement)

The wheel motors provide the robot's main locomotion capabilities:

- Forward and backward driving
- Turning using differential wheel speeds
- Supporting overall body balance
- Smooth, controllable torque for predictable acceleration

The wheel motors receive:

- Speed targets from the microcontroller
- Encoder feedback for closed-loop control

The locomotion system also feeds back motion data used by stability control loops. In order to acquire the required motor feedback, the motor must have high quality encoders and sensors to sense position and rotation speed. A high-quality motor controller must be used in conjunction to ensure timely data and proper feedback / control occur.

#### Stability and Control

Stability is managed through both mechanical design and software control:

- Keeping the center of mass inside the chassis footprint for baseline stability
- Utilizing a locked third caster wheel to maintain balance
- Using the IMU and anti-tip control to correct small tilts, maintain consistent wheel speeds, and smooth motion

These features ensure the robot remains stable during demonstrations and movement.

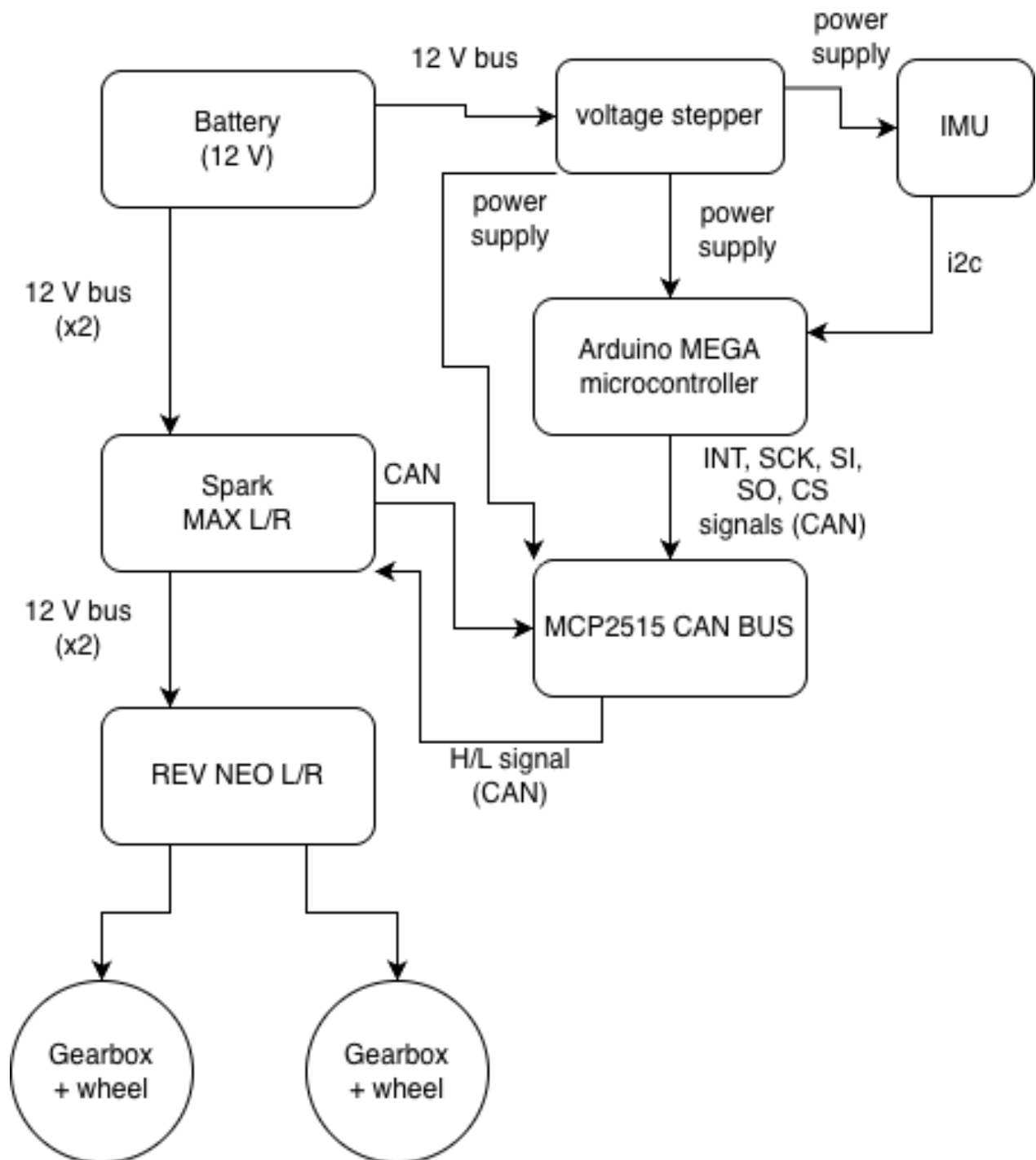
#### Legs

Each leg is both structural and aesthetic and includes:

- A hip joint for future motor integration purposes
- A knee joint for future motor integration purposes
- A wheel assembly with its own motor bracket

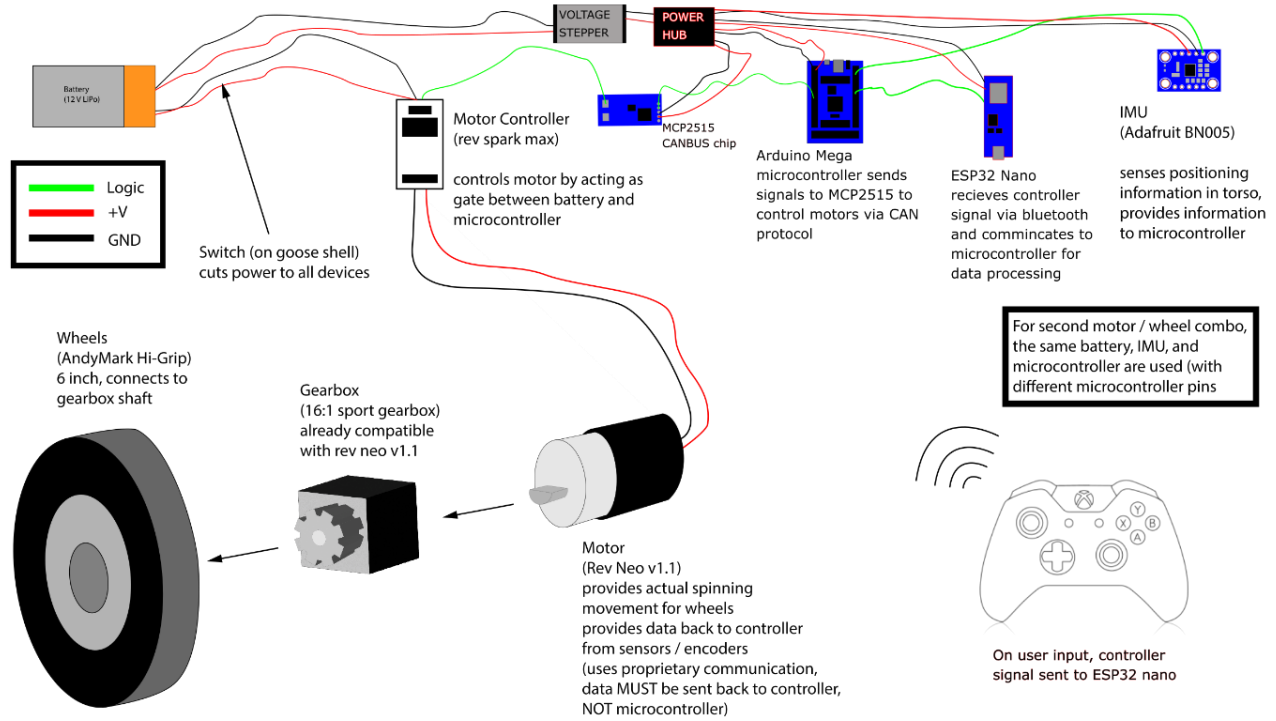
The legs must support the full weight of the robot while allowing fluid joint motion if needed. The mechanical locking feature is planned so the robot can switch into a rigid, stable standing mode only.

#### 4.1.1 Block Diagram

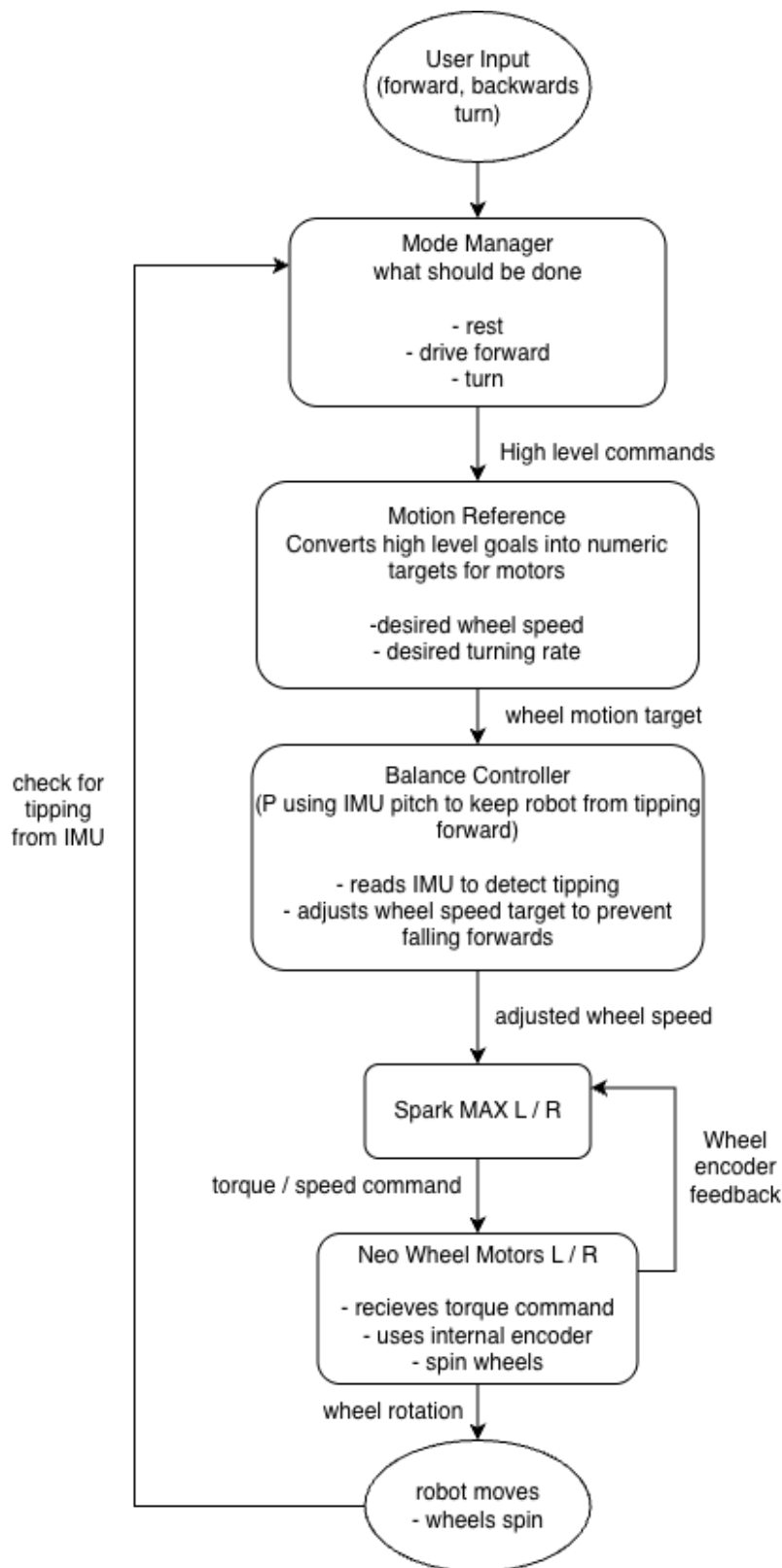




## 4.1.2 Motor Diagrams



### 4.1.3 Motor Flowcharts



## 4.2 ELECTRICAL DESIGN

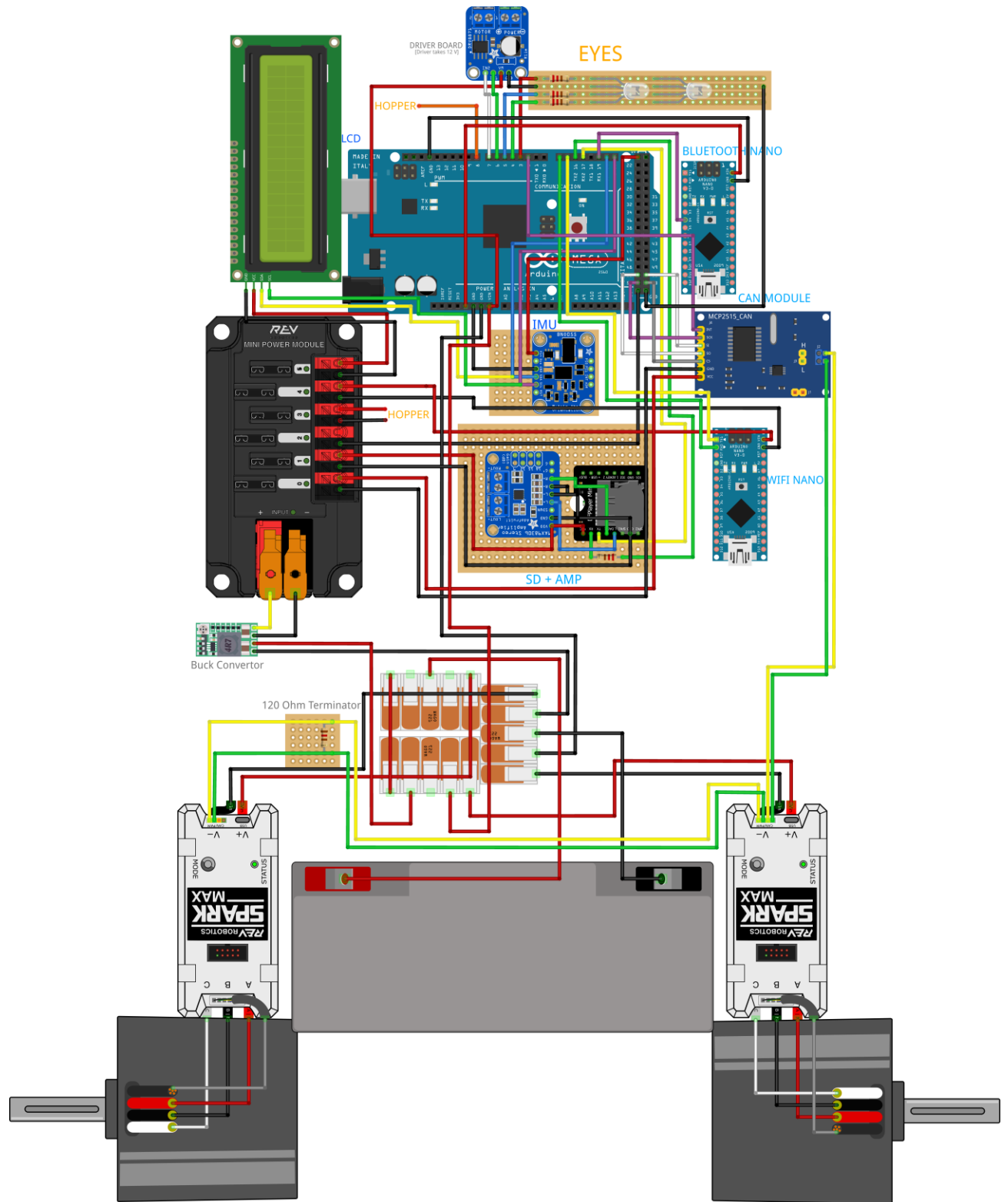
### Power Distribution

Power flows from a single 3S LiPo battery (9.6V empty, 12.6V full nominal). The battery connects directly to the Wago Power Distribution Hub which splits power between each Spark MAX → motors, Arduino Mega, and Voltage Step Down Converter to 5 V → REV Mini Power Module. Mini Power Module Distributes Power as follows:

| Port            | Peripheral                           |
|-----------------|--------------------------------------|
| 0               | MCP2515 CAN Shield                   |
| 1               | DFPlayer Mini + Amplifier Board      |
| 2 (Ground Only) | Arduino Mega (Needed for common GND) |
| 3               | Hopper (Unused)                      |
| 4               | Soundboard Arduino Nano (WiFi)       |
| 5               | 16x2 LCD Screen                      |

Keep in mind that all grounds must connect together (battery, 5V rail, every board's GND).

# Fritzing Diagram

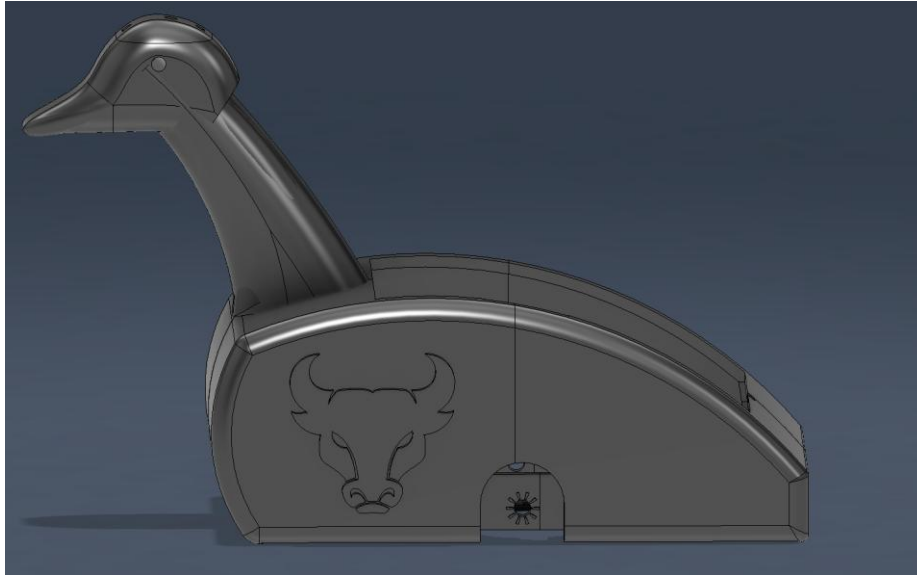


fritzing

## 4.3 BODY STRUCTURE

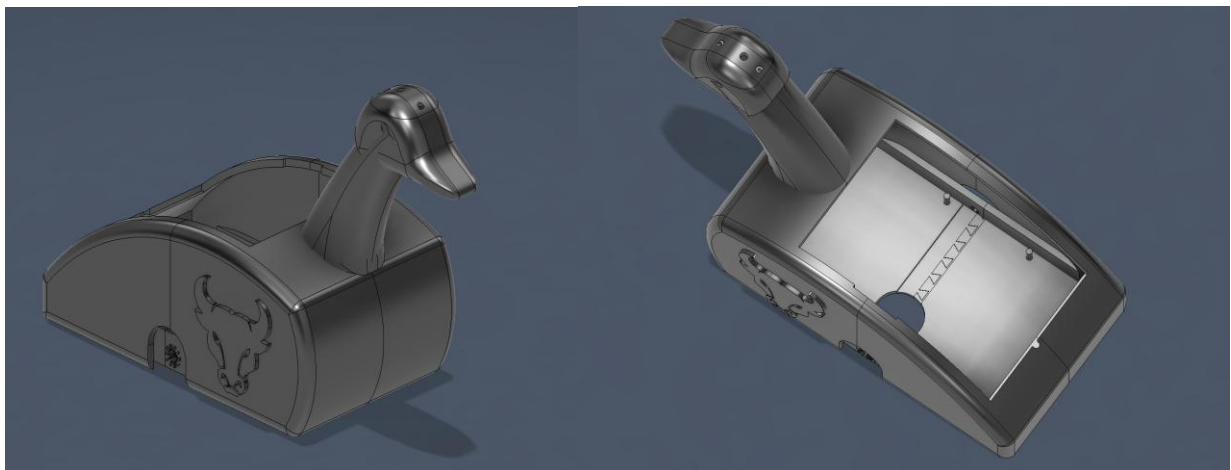
### Global Mass Constraint

- Total mass limit: 18kg
- All subsystem masses must remain below this limit



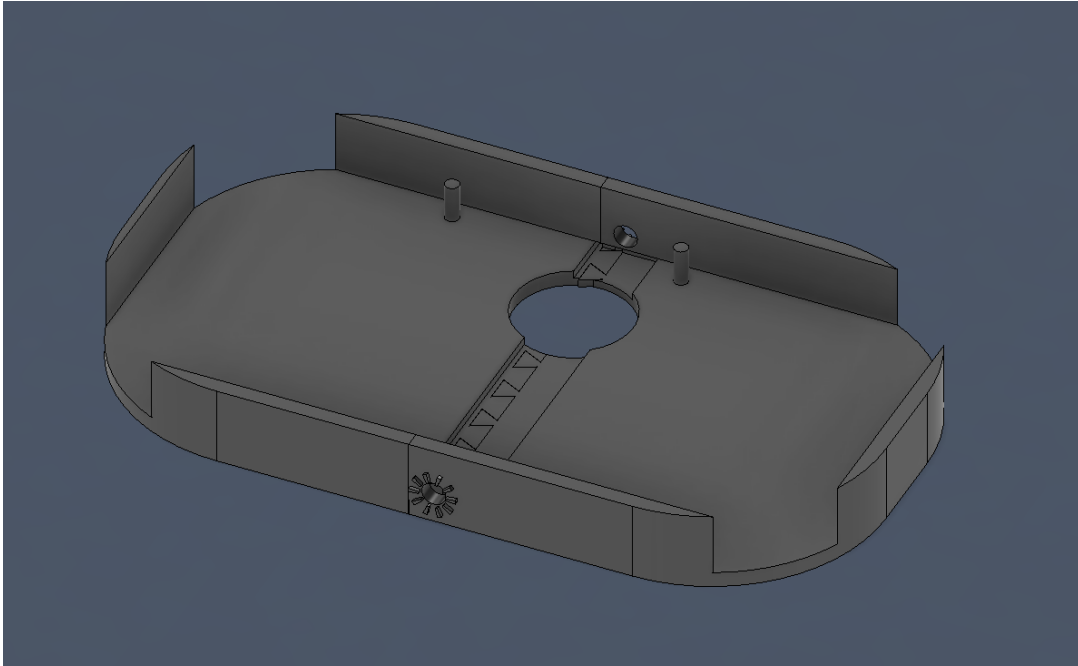
### Body – Outer Shell

- Hollow stretched-ellipsoid geometry
- Dimensions: ~ 405mm (length), 240mm (width), 360mm (height bottom - top of head)
- Wall thickness: 8mm
- Mass ceiling: 3 kg
- Purpose: goose appearance and protective enclosure
- Open back to showcase hardware and doubling as ventilation.



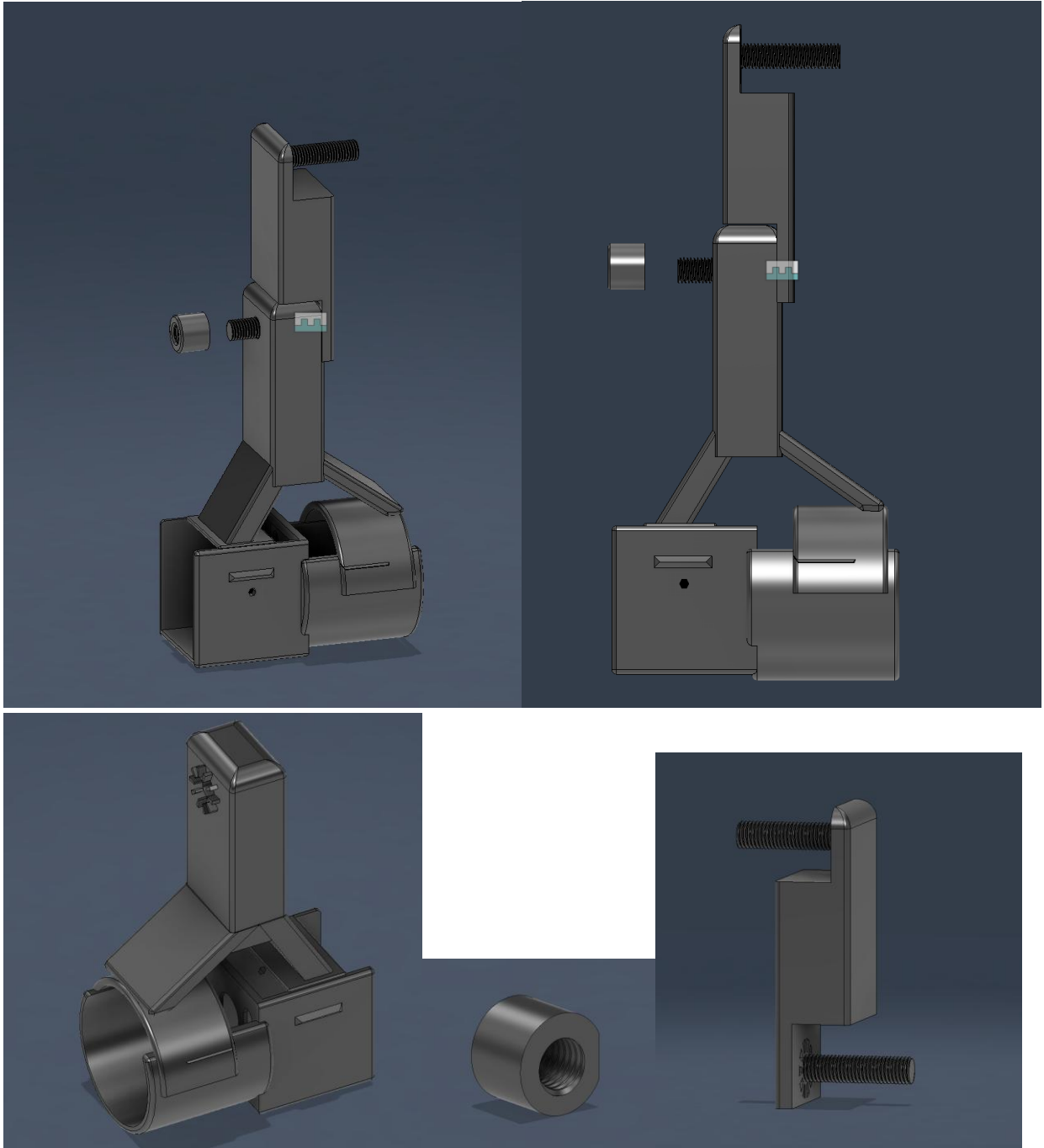
## Body – Chassis

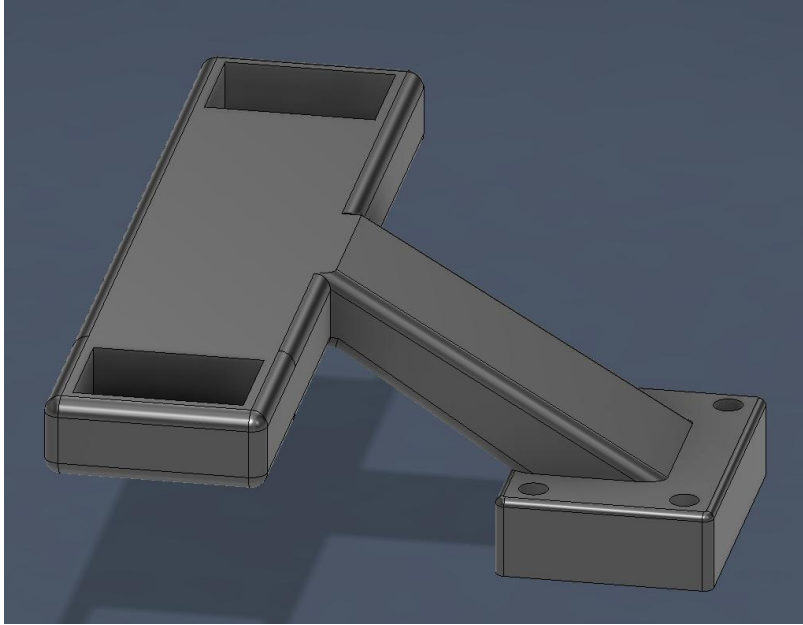
- Rectangular load bearing internal structure
- Dimensions: 384mm (length), 220mm (width), 40mm (height)
- Mass ceiling: 3 kg
- Purpose: Houses battery, electronics, hopper, interfaces with legs, shell and neck through mounting points



## Legs

- 3 leg components, two major legs and one training wheel leg.
- Each leg is about ~ 275 mm total in length when combined
- When separate lower leg ~ 195mm, upper leg~ 121mm
- Mass ceiling: 0.5kg per leg
- 2-DOF (hip + knee) -> requires motor integration and PID control for enhanced control (at the time of this document there are no hip or knee motors integrated)
- Rigid, fixed configuration is the current default





## Materials

- **Body/Shell/Chassis/Legs:** PLA
- Mass estimation:

*(Reference material mass equations in Appendix)*

*(Calculations also in Appendix)*

## Center of Mass (CoM)

- CoM must remain inside the chassis footprint
- Forward/back tolerance:  $\pm 5\text{cm}$  from center
- CoM height:  $\leq 20\text{ cm}$

*(Reference CoM equations in appendix)*

## Stability Limit

- Stable as long as CoM is inside Chassis during acceleration, braking and turning
- The design is intended for flat indoor floors and is not meant for steep slopes or uneven terrain

## Mobility Goals

- The legs support the load and connect the chassis to the wheels
- The leg design must stay rigid and stable while allowing forward, backward, and turning motion



## Leg Joint Configuration

Designed for 2-Degrees-of-Freedom (DoF).

- 2-DOF:
  - o Two joints per leg (hip +knee)
  - o Allows limited articulation or small posture adjustments
  - o More complex and must remain stiff under load

## 4.4 PERIPHERALS

There are 5 peripherals included in the ROBO-GOOSE. These include a wireless controller to move the goose and activate other peripherals, a candy hopper, RGB LED eyes, a speaker to honk, make funny sounds and play some songs, and a spinning hat.

### Controller

The controller allows for the robot to be maneuvered wirelessly over a Bluetooth connection. The controller will be handheld. This is done through an Arduino Nano ESP32 which then connects to the Arduino Mega. The Arduino Mega then relays the received controller input to the motor controller.

The controller implements the ability to control the wheels, pair to the robot, change the coloration of the RGB eyes, and play a variety of songs and sounds. The songs and sounds can be modified by using a computer to interface with the micro SD card located inside of the DFPlayer Mini.

The controller's inputs are harvested using the Bluepad32 library

### Bluepad32 and Technical Information

Bluepad32 is an open-source library that allows for the ESP32 to be a Bluetooth host capable of pairing with most modern game controllers. Compatible controllers include Xbox, PlayStation, and Nintendo Switch, among others.

Most information regarding Bluepad32 and its technicalities can be found at their website:

<https://bluepad32.readthedocs.io/en/latest/>

Upon boot the Nano will initialize the BT stack and enter discoverable mode, meaning it will automatically begin searching for compatible controllers. To pair the Xbox controller, simply hold the pairing button on the back of the controller by the charging port. Once the center "X" button blinks white, it will begin searching for compatible devices. Once connected, the button will then glow solid white.

Upon input, Bluepad32 fires a callback, stuffing the values obtained from the controller into global variables. The loop function then packs the most recent values of these variables into an 8 field CSV block, which is then sent to the Mega over serial.

The layout of the CSV is as follows:

lx, ly , rx, ry, brake, throttle, buttons, dpad

| # | Field    | Source            | Range       | Notes                                                                                                            |
|---|----------|-------------------|-------------|------------------------------------------------------------------------------------------------------------------|
| 1 | lx       | Left stick X      | -512 to 512 | Negative = left, positive = right (unused)                                                                       |
| 2 | ly       | Left stick Y      | -512 to 512 | Negative = up (forward), positive = down (reverse)                                                               |
| 3 | rx       | Right stick X     | -512 to 512 | Negative = left, positive = right (used for steering)                                                            |
| 4 | ry       | Right stick Y     | -512 to 512 | Negative = up, positive = down (unused)                                                                          |
| 5 | brake    | LT analog trigger | 0 to 1023   | Used as a modifier flag                                                                                          |
| 6 | throttle | RT analog trigger | 0 to 1023   | Triggers blue RGB eyes                                                                                           |
| 7 | buttons  | Button bitmask    | 0 to 65535  | X plays random honk, Y plays random noise, A is green eyes, B is red eyes, see reference for further information |
| 8 | dpad     | D-pad bitmask     | 0 to 15     | Primarily used for playlist management                                                                           |

For the buttons and d-pad functionality, a bitmask is used where each bit represents one button. Multiple can be used simultaneously by OR-ing them together

| Bit | Decimal | Constant | Button            |
|-----|---------|----------|-------------------|
| 0   | 1       | BTN_A    | A                 |
| 1   | 2       | BTN_B    | B                 |
| 2   | 4       | BTN_X    | X                 |
| 3   | 16      | BTN_Y    | Y                 |
| 4   | 32      | BTN_LB   | Left Bumper       |
| 5   | 64      | BTN_RB   | Right Bumper      |
| 6   | 128     | N/A      | N/A               |
| 7   | 256     | N/A      | N/A               |
| 8   | 512     | BTN_LS   | Left Stick click  |
| 9   | 1024    | BTN_RS   | Right Stick click |
| 10  | 2048    | N/A      | N/A               |

Examples:

- If buttons = 16 -> only LB pressed
- If buttons = 17 -> both LB and A are pressed (16 + 1)

Bits 6,7, and 10+ are reserved but unused within our implementation, refer to the Bluepad32 docs and source code for which buttons they apply to should you wish to implement additional buttons. Keep in mind, we employ the left trigger as a modifier button which can be used to effectively double the number of potential inputs.

The same bitmask functionality applies to the D-pad:

| Bit | Decimal | Constant   | Direction |
|-----|---------|------------|-----------|
| 0   | 1       | DPAD_UP    | Up        |
| 1   | 2       | DPAD_DOWN  | Down      |
| 2   | 4       | DPAD_RIGHT | Right     |
| 3   | 8       | DPAD_LEFT  | Left      |

Currently, our code checks for D-Pad input individually; if you would like to add support for diagonals, these handlers must be adapted.

Example frame:

0,-450,250,0,0,0,2

This CSV frame would do the following:

lx = 0, ly = -450 -> Robot will move forward at ~90% speed

rx = 250, ry = 0 -> Robot will move right at about half deflection

brake = 0, throttle = 0 -> no triggers are pressed

buttons = 0 -> no buttons are pressed

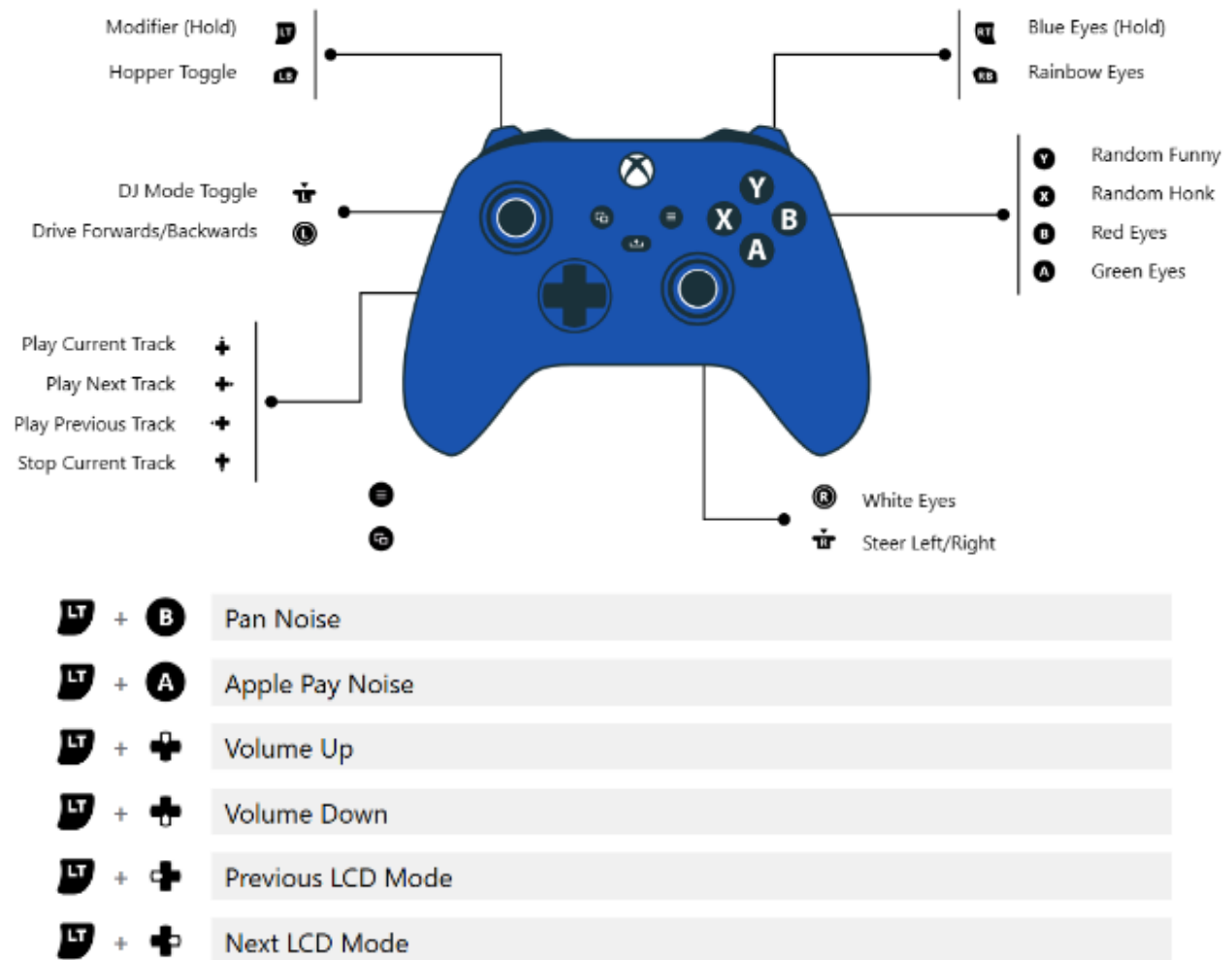
dpad = 2 -> D-pad down pressed

Once the Mega receives the CSV packet, it parses the fields and performs the corresponding action via various handlers.

Keep in mind that the robot is limited in the code to only ever operate at 50% motor speed. This is done for primarily safety purposes as the faster it is the more difficult to control it becomes.

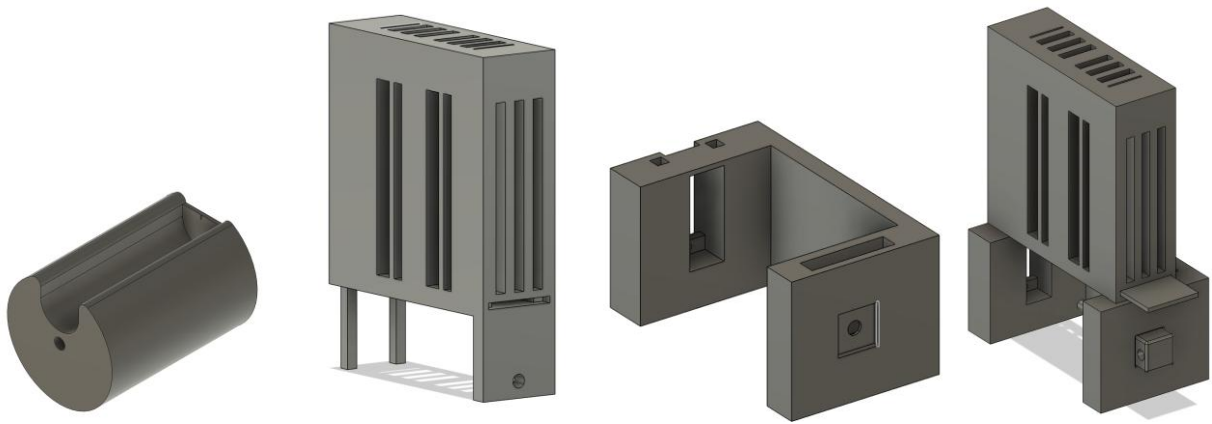
This can be changed via a variable, but be careful when adjusting this!

## Controller Diagrams



## Candy Hopper

The candy hopper dispenses a single piece of candy when signaled by the controller. It is driven by a standard hobby servo on Mega pin 8 that rotates a magazine-fed dispenser. The servo connects to 5V, ground, and a PWM signal from the Arduino. Two positions are defined in code: `HOPPER_CLOSED` (75°) and `HOPPER_OPEN` (170°), toggled by the LB button. The candy hopper is currently non-operational and must be updated to prevent the candies from getting stuck. A semi-successful design is shown below. This design features a magazine of candies, which is slotted into a dispenser. When the lid to the magazine is removed, the dispenser catches a tootsie roll and rotates towards the wall to release it. Below is the rotating dispenser, the magazine, the base, and the full system (with the dispenser support and lid).



## DFPlayer Mini Audio + Amp / Speaker

The [DFPlayer Mini](#) reads MP3s off a microSD card and plays them through a built-in DAC. We feed its output into a MAX98306 amp, then to the speaker.

Library: dfrobot/DFRobotDFPlayerMini. Volume range is 0–30, the code starts at 30 and lets the user adjust with LT+DPad.

### SD card folder layout:

| Folder | Purpose                             | Track Count       |
|--------|-------------------------------------|-------------------|
| 01     | Startup sounds (random on boot)     | 4                 |
| 02     | Music tracks (D-pad cycling)        | 29                |
| 03     | Annoying sounds (Y button + LT+A/B) | 17                |
| 04     | Honks (X button)                    | 17                |
| 05     | DJ Goose intros                     | 6 specific tracks |
| 06     | DJ on/off toggle SFX                | 2                 |
| 07     | Death sounds                        | 4                 |

DFPlayer is capable of reading up to 99 folders on an SD card, with each containing 255 .mp3 files. Filenames must be 3 digits with leading zeros ([001.mp3](#), [002.mp3](#), ...). The DFPlayer is picky about this and silently fails if the names are wrong. Filenames can be labeled with the following format: 005\_songname.mp3 OR 005.mp3. Folders cannot use this format and must be in the following format: 02/, 04/, etc.

DJ mode plays a corresponding "intro" track from folder 05 before the music starts. The transition uses a fixed 5-second timer instead of detecting end-of-file as ([DJPlayerPlayFinished](#) had false positives during testing). This was a fun feature that was completely hard coded and a lot of work to maintain.

The speaker emits sound when signaled by the controller. The Rev Mini Power Module supplies the [MAX 90306 Class D Audio Amplifier](#) and DFPlayer Mini with 5V and GND connections.

## 16x2 I2C LCD

The LCD provides live diagnostic readouts during operation. It's a standard 16x2 character display with an I2C backpack, sharing the I2C bus on Mega pins 20 (SDA) and 21 (SCL) with the BNO055 IMU. The two devices use different I2C addresses so they coexist without conflict.

Library: marcoschwartz/LiquidCrystal\_I2C. The code initializes the LCD at address `0x27`, which is the most common factory address for the I2C backpack. It should be noted that some backpacks ship at `0x3F` instead.

The LCD has five display modes, cycled by holding LT and pressing D-pad left or right:

| Mode | Display                                               |
|------|-------------------------------------------------------|
| 0    | Left and right motor RPM                              |
| 1    | Left and right motor temperatures                     |
| 2    | Battery voltage, battery percentage, current volume   |
| 3    | Current music track, DJ mode state, current volume    |
| 4    | Live IMU pitch and forward tilt (for anti-tip tuning) |



Modes 0 through 3 refresh every 500ms to keep the main loop responsive. Mode 4 refreshes every 150ms so you can watch tilt values change in real time while tuning anti-tip constants. The faster refresh on mode 4 steals enough loop time to make driving feel noticeably less responsive, so switch off mode 4 when actually driving the robot.

Display content is built into a 17-byte character buffer per line and padded to exactly 16 characters before pushing to the LCD. This prevents ghost characters from previous mode contents lingering when switching between modes.

The LCD will continue to operate even if the IMU is disconnected. Mode 4 will simply display "IMU NOT FOUND" until the sensor is wired back in.

## BNO055 IMU

The BNO055 is a 9-DOF inertial measurement unit (accelerometer, gyroscope, and magnetometer with onboard sensor fusion) used to provide a stable pitch angle in degrees for the anti-tip system. Without the IMU, anti-tip is automatically disabled, and the goose will drive without forward-lean correction.

### Hardware

The BNO055 shares the I2C bus with the LCD on Mega pins 20 (SDA) and 21 (SCL). The IMU's I2C address is `0x28`, distinct from the LCD's `0x27`. Power and ground come from the same 5V rail as the LCD.

### Library

The libraries used are `adafruit/BNO055` along with `adafruit/Adafruit Unified Sensor`. The code uses the IMU's onboard fusion in NDOF mode, which combines the three sensors automatically and provides absolute orientation. Only the pitch axis is read by the firmware. The IMU also uses an external crystal for more stable readings, enabled with `bno.setExtCrystalUse(true)` during setup.

The code reads pitch from `event.orientation.y`. Which axis corresponds to physical pitch depends entirely on how the sensor is mounted in the goose. If the IMU is rotated 90 degrees from its expected orientation, pitch will show up on the x axis instead. If the readings look like roll or yaw instead of pitch, swap the axis in `readIMU()` and reflash.

### Anti-Tip System

The IMU drives a three-stage anti-tip system designed to keep the goose upright during hard braking. All three stages compare the live pitch reading to the calibrated resting pitch (`TILT_RESTING`) to compute "forward tilt": how far the goose has leaned past level. Each stage activates at a different forward-tilt threshold and does something different.

**Stage 1: Brake Softening.** When forward tilt crosses `TILT_THRESHOLD` (3 degrees from resting), the firmware starts reducing the rate at which the throttle ramps down during braking. The reduction scales linearly with how far past the threshold the goose has leaned — at `TILT_THRESHOLD` braking is at full strength, and by `TILT_MAX` braking is reduced to 10% of normal. This is passive: it doesn't actively drive the motors, it just eases off the brakes so the goose's forward momentum carries it back down rather than pitching forward. Most "almost tipped" recoveries happen entirely in Stage 1 and the driver never notices.

**Stage 2: Active Catch Boost.** If forward tilt exceeds `TILT_MAX`, brake softening alone isn't enough and the goose needs an active save. Stage 2 adds forward duty cycle to both motors proportional to how far past `TILT_MAX` the tilt has gone (controlled by `CATCH_GAIN`), capped at `CATCH_MAX`. This drives the wheels forward to slide them under the falling goose, the same instinct a person has when they lean forward and step into it to catch themselves. The catch boost works regardless of stick input, so a stationary goose tipped forward by a kick or push will still try to save itself. The boost is applied after tank-drive mixing and after the `MAX_DUTY` cap, so the save move can briefly exceed normal speed limits when needed.

**Stage 3: Death Mode (Failsafe).** If pitch goes outside the safe range entirely, the goose has tipped over or crashed and the firmware gives up trying to save it. Death mode triggers in two ways:

- **Forward tip:** pitch drops below `DEATH_PITCH`. The goose is face-down.
- **Backward tip:** pitch rises above `DEATH_PITCH_BACK`. Usually from crashing into something and rocking backward.

When any of these triggers, the firmware immediately cuts motor duty to zero, zeros out the throttle and steering ramp state (so the goose doesn't lurch on revive), and plays a random death sound from folder 07 on the SD card. The LCD displays "GOOSE DEAD" if mode 4 is active. The goose stays dead until pitch returns to a sane upright range (between `DEATH_RECOVERY_PITCH` and `DEATH_RECOVERY_BACK`) — typically requires a human to pick it up and right it.

All five thresholds (`TILT_RESTING`, `TILT_THRESHOLD`, `TILT_MAX`, `DEATH_PITCH`, `DEATH_PITCH_BACK`) are constants at the top of `main.cpp` and tunable without restructuring code. Tuning guidance is in Section 7.

**Calibration:** the resting pitch value is stored in the `TILT_RESTING` constant at the top of `main.cpp`. This represents the pitch the IMU reads when the goose is sitting upright on a level surface. To recalibrate:

1. Power on the goose and let it sit level on the floor for a few seconds so the IMU settles
2. Switch the LCD to mode 4 (LT + D-pad right, four times from mode 0)
3. Read the pitch value displayed on line 1
4. Update `TILT_RESTING` in `main.cpp` to that value
5. Reflash

Recalibrate any time the IMU is physically remounted, the chassis is rebuilt, or the goose is given new feet that change its resting angle.

The IMU system also has a sanity range check built in. If pitch reads outside the range of -90 to +90 degrees, the firmware assumes the sensor returned garbage data and ignores it. This prevented a bug where the IMU glitched after a crash and caused the catch-boost logic to drive the motors forward indefinitely.

## Built-in Web Server

The goose also features its own integrated web server hosted entirely on an additional Arduino Nano ESP32. The server hosts an app that allows the user to connect with a mobile device to play music, sounds, and manage telemetry on the goose.

### Network Details:

- SSID: **Goosebot**
- Password: **pizzamov!e** (defined as a string in the Nano code; change and reflash to update)
- URL: **goosebot.io** (resolved locally by the Nano acting as a DNS server)

First the Nano acts as a WiFi access point, broadcasting beacon frames with the SSID “**GooseBot**”. Upon attempting to connect the user will be prompted with a password request. This password is defined in the code as a string and can be updated by changing said string and re-flashing the Nano code. Upon successful password entry, the user’s device and the Nano will perform a standard WPA2 4-way handshake to authenticate the user. Once connected, all packets are encrypted with AES-128.

Once connected, the Nano also acts as a DNS server. When a connected device attempts to resolve any IP address, the Nano responds with its own AP IP, automatically rerouting the device to the web app (Note: if your device does not automatically open the app upon connection, simply enter the URL manually: **goosebot.io**).

Once you’ve arrived, the Nano hosts its own HTTP server. This is what the actual app runs on. All html is written directly to the nano, so should any of the website need to be changed, edit this code. Telemetry events and song state are served to the user via GET request from the server. Change in state (like when a song is paused, played, skipped, volume changed, etc.) is then sent back to the Nano via POST request. Data is then transmitted back and forth between the Nano and Mega over TX/RX serial3.

Keep in mind the network is not connected to the outside internet in ANY way, meaning your device will most likely not have any access to anything requiring an internet connection.

## RGB LED Eyes

Two RGB LEDs wired in parallel through 220  $\Omega$  resistors to PWM pins 3 (R), 4 (G), and 5 (B) with common cathode (to ground). The code drives them with `analogWrite()` utilizing HSV to RGB conversion for smooth color blending for rainbow mode, where colors cycle. Also lights up as solid colors depending on user input. Eyes share a protoboard with Spinning Hat's driver board.

Three modes:

- Solid color with breathing/blink animation (default)
- Rainbow cycle (continuously rotates hue)
- Off (only at boot, eyes light up as startup sound plays)

The breathing animation is a 4-state machine: hold bright  $\rightarrow$  fade to dim  $\rightarrow$  hold dim  $\rightarrow$  fade to bright. Timing constants (`BLINK_HOLD_BRIGHT`, `BLINK_CLOSE_TIME`, `BLINK_HOLD_DIM`, `BLINK_OPEN_TIME`) and the dim level (`BLINK_DIM_LEVEL`, where 0.0 = off and 1.0 = same as bright) are tweakable at the top of `main.cpp`. `RAINBOW_SPEED` controls how fast the rainbow cycles.

## Spinning Hat

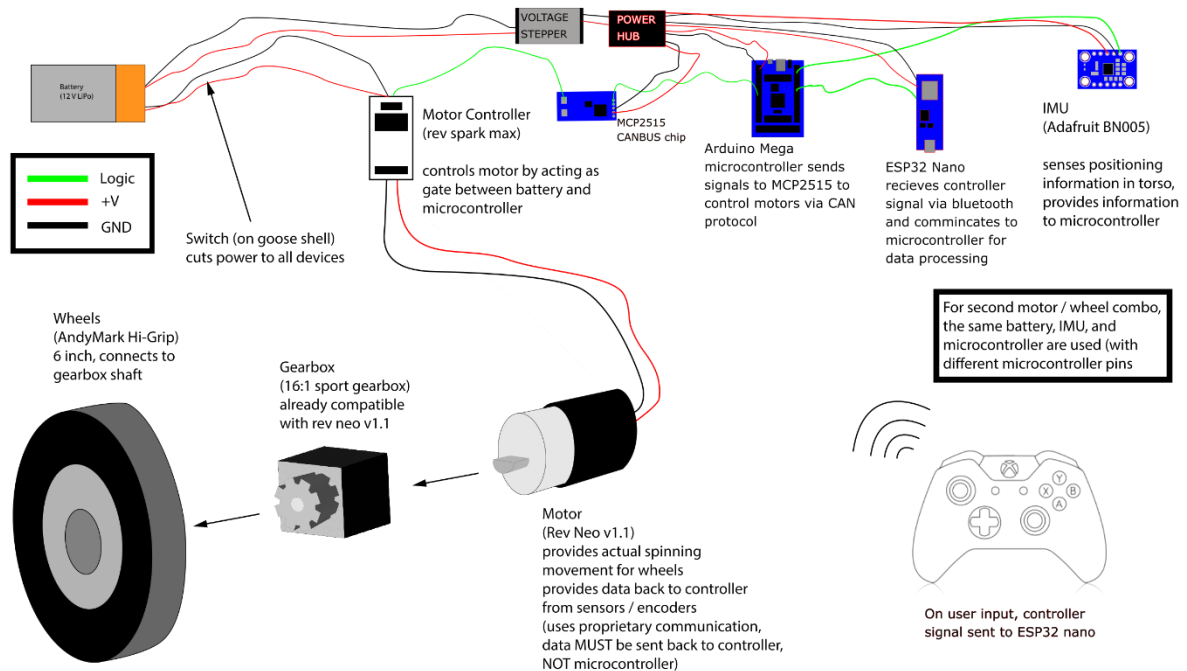
The spinning hat uses a hobby motor to rotate a fan blade at the top of the robot's head when signaled by the controller. The motor requires a driver and is controlled by a PWM signal from the Arduino. The motor can be placed in the head by unscrewing the lid and screwing the motor into the mount (pictured below). Once the lid is replaced, the fan blade can be connected to the motor. The wires for the motor are fed through the neck. We used the DRV8871 Driver Breakout Board which takes a 12V (not 5V) input with IN1 as speed, and IN2 for direction.



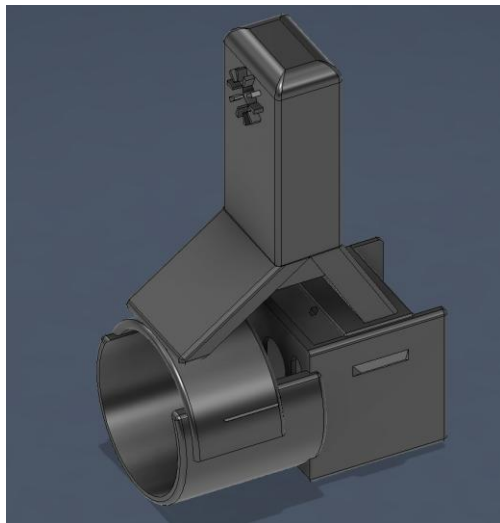
## 5. ASSEMBLY

This section is a step-by-step guide on how to assemble the ROBO-GOOSE (NEMESIS) with wiring and assembly schematics.

The motors are connected to the system power and logic as follows in the diagram (exact electronic connections can be seen in previous fritzing diagrams):



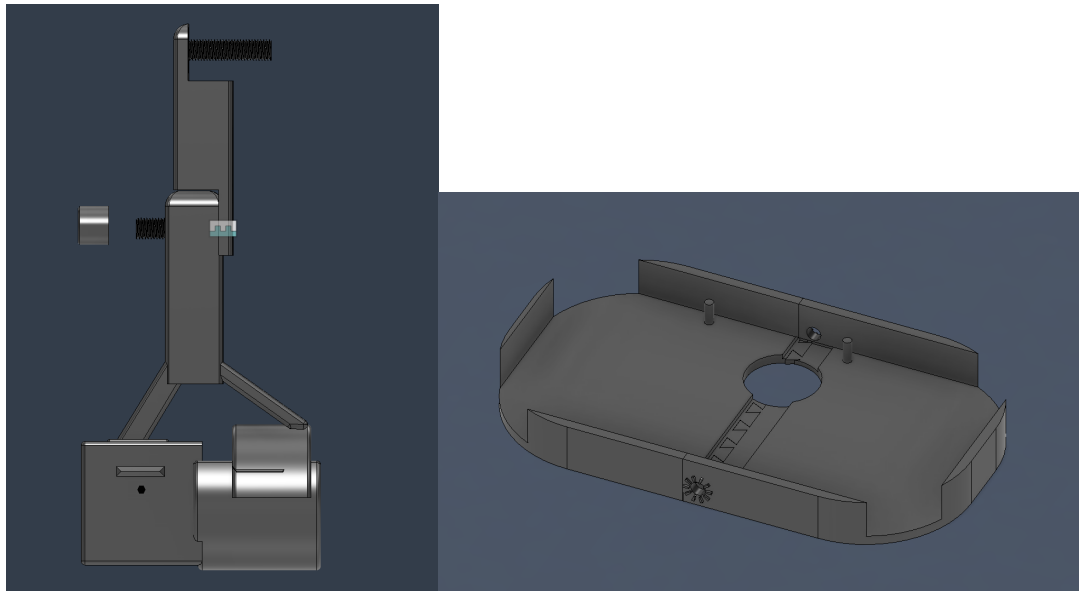
The motors are installed into the foot and can be conceptualized by looking at the CAD model for the foot as reference:



The motor is inserted into the round side of the foot below, with pinion passing through the hole in the flat piece of plastic between the round and square sides. The gearbox is inserted into the square side of the foot, with the pinion pointing outwards. The motor pinion should protrude through the wall and into the gearbox. Screws provided with the gearbox should reach through the gearbox, through the smaller side holes in the center flat piece of plastic, and into the peripheral screw holes on the face of the motor. Wires can be run up the leg to the chassis. Gearboxes are mounted to the flat rectangular piece of the leg above the foot piece via zip ties.

The gearbox can be attached to the wheel using a single 10-32 screw and washer. The screw goes through the washer and the center hole of the wheel on one side and into the hole in the pinion of the gearbox on the other side.

The training wheel leg slides over the motor mounts, connecting each of the legs. The legs are affixed to the torso chassis via a screw and bolt system that can also be seen looking at the CAD files:



The same screw and bolt system seen on the knee of the leg system is used to affix the leg to the chassis. The top screw of the leg is inserted into the hole in the side wall of the chassis. A plastic nut is used inside the chassis to secure the screw.

The shell and head, though printed as separate pieces, are connected using two part plastibond epoxy in the shape seen below. The heavier lines indicate seams that need to be glued.



The shell is affixed to the chassis using screws and bolts as seen above.

Wires can be run up the hollow neck of the goose and into the brain to reach the servo for the spinning hat. Details about mounting of that servo can be found in the Spinning hat section seen prior in this document.



## 6. BUILD & FLASH

### 6.1 REQUIRED SOFTWARE

- **VS Code** (or any editor) with the **PlatformIO IDE extension**
- **Arduino IDE** is used for ESP32 board packages and DFU drivers, used for both the Bluetooth and WiFi Arduino Nanos. Bluepad32 is only usable with Arduino IDE. The WiFi Nano was uploaded using Arduino IDE as well.
- **Git** for cloning and version control

### 6.2 SETTING UP PlatformIO IN VSCODE

If this is your first time using PlatformIO, here's the full setup process:

1. **Install VS Code** if you don't already have it.
2. **Open the Extensions panel** in VS Code (Ctrl+Shift+X or click the squares icon in the left sidebar).
3. **Search for "PlatformIO IDE"** and click Install. It's a fairly large download (a few hundred MB) because it bundles its own Python environment and toolchains. First-time install can take 5-10 minutes.
4. **Restart VS Code** when prompted. The PlatformIO icon will appear in the left sidebar (an alien-head logo).
5. **Clone this repo** somewhere on your machine, then open the folder in VS Code (File -> Open Folder).
6. **Wait for PlatformIO to initialize.** It will detect `platformio.ini`, fetch all the library dependencies listed there, and download the AVR toolchain if you don't have it. This first-time setup also takes a few minutes.
7. **Verify the build environment** by clicking the PlatformIO icon and checking that the `[env:mega]` environment appears under "Project Tasks". If it does, you're set up.

## 6.3 PlatformIO CONFIGURATION

The project uses `platformio.ini` for build config. The important parts for building with the Mega:

```
[env]
monitor_speed = 115200
lib_ldf_mode = deep+
lib_deps =
    autowp/autowp-mcp2515@^1.3.1
    dfrobot/DFRobotDFPlayerMini@^1.0.6
    arduino-libraries/Servo@^1.2.2
    marcoschwartz/LiquidCrystal_I2C@^1.1.4
    adafruit/Adafruit BNO055@^1.6.3
    adafruit/Adafruit Unified Sensor@^1.1.14
    SPI
    Wire
build_flags = -I${PROJECT_INCLUDE_DIR} -I${PROJECT_INCLUDE_DIR}/low_level -
               I${PROJECT_INCLUDE_DIR}/motors -DCANCONTROL_MAIN
test_framework = unity
test_filter = test_embedded
test_build_src = yes

[env:mega]
framework = arduino
platform = atmelavr
board = megaatmega2560
build_flags = ${env.build_flags} -DEXAMPLE_BASIC
```

The `-DEXAMPLE_BASIC` flag is required because the project was forked from a multi-example template. Without it, the linker complains that `setup()` and `loop()` are missing, they're guarded by `#ifdef EXAMPLE_BASIC`.

## 6.4 BUILDING & FLASHING

Make sure no serial monitor is open in another window when uploading. Windows holds the COM port and the upload will fail.

Click the PlatformIO icon on the left sidebar, drop down the mega project task, and upload to flash onto the Arduino Mega. Upload and Monitor to upload and read serial output.

## 6.5 DEBUGGING VIA SERIAL

Set `SERIAL_ENABLED = true` at the top of `main.cpp` and reflash. You'll get a telemetry line every 250ms in the serial monitor showing every state in the system: stick values, motor duty cycles, RPMs, temperatures, voltage, current track, DJ mode, hopper state, IMU pitch, and packet RX counters. Turn this off again before deploying ROBO-GOOSE (serial prints are slow and add per-loop overhead).

For deeper CAN debugging, set `DEBUG_RAW_BYTES = true` to see raw hex dumps of every status frame. This is mainly useful when adding support for new Spark MAX status frames. Ensure this is turned off before deploying ROBO-GOOSE (this adds *extreme* overhead on loops).

## 6.6 SETTING UP Bluepad32

The Xbox controller bridge runs on an Arduino Nano ESP32 using the Bluepad32 library. Setup is more involved than a typical Arduino project because Bluepad32 ships its own board package, and the Nano ESP32 has a finicky bootloader. Read this whole section before starting.

### 6.6.1 Install the Bluepad32 board package

Bluepad32 isn't a normal library, it's a full board package that replaces the standard ESP32 one for this Nano. Follow the official tutorial at [bluepad32.readthedocs.io/en/latest/plat\\_arduino](https://bluepad32.readthedocs.io/en/latest/plat_arduino) to add it through the Arduino IDE Boards Manager. The tutorial walks through adding the Bluepad32 board URL to your Arduino IDE preferences and installing the package.

### 6.6.2 Configure board settings in Arduino IDE

Once the Bluepad32 boards are installed, in the Arduino IDE:

- **Tools → Board → esp32\_bluepad32 → Arduino Nano ESP32**
- **Tools → Pin Numbering → GPIO number** (NOT "By Arduino pin"). The pin numbers in our code reference GPIO numbers directly, so the wrong setting will cause every pin to map incorrectly.
- **Tools → USB Mode → Debug mode**. There's a known bug in the Bluepad32 library where serial output doesn't work correctly in the default USB mode.

### 6.6.3 Upload the code (Nano bootloader mode)

The first time you try to upload, you'll almost certainly get an error 74 along the lines of:

```
dfu-util: No DFU capable USB device available
```

This is the Nano ESP32 not being in its bootloader mode. To fix it:

1. **Double-tap the reset button** on the Nano ESP32. The timing matters. **There needs to be a brief pause between presses** (roughly half a second). Too fast and it won't register as a double-tap; too slow and the first press will just reset normally.
2. The onboard LED should start a **slow green "breathing" effect**. That's the indicator you're in bootloader mode.
3. **Upload the code while the LED is breathing**. The breathing effect will stop once the upload completes.
4. **Tap reset once more** to start the freshly-flashed firmware.

If you don't get the breathing LED, try the double-tap again. This bootloader behavior is finicky and sometimes takes a few tries.

### 6.6.4 Pair the controller

Once the code is running, the Nano will automatically enter discovery mode and start looking for a Bluetooth controller. To pair:

1. Hold the **sync button on the back of the Xbox controller** (next to the charging port) until the Xbox logo on the front starts flashing rapidly.
2. After a few seconds the Nano will pick it up and the controller's light will go solid.
3. From this point on, the Nano sends CSV input packets to the Mega over the serial line at 9600 baud and the goose is drivable.

### Controller Troubleshooting

If pairing fails or the goose isn't responding to controller input:

- Check that the Xbox controller has charged batteries
- Make sure the Nano is powered on and has a solid Bluetooth connection (LED behavior varies by firmware version, but a connected state should be obvious)
- Verify the TX line from the Nano is wired to the Mega's Serial1 RX (pin 19) - the connection is one-way, so RX from the Nano isn't needed
- Confirm both boards share a common ground

## 7. QUICK START GUIDE

If you're picking up this project and want to make your first change:

1. **Pair the controller.** Power on the goose. The Bluetooth Nano ESP32 will auto-discover and pair with any Xbox controller in pairing mode (hold the sync button until the light flashes rapidly).
2. **Open the project in VS Code.** With the PlatformIO extension installed, it will auto-detect `platformio.ini` and start fetching libraries on first open.
3. **Make a small change.** Open `src/main.cpp`, find `MAX_DUTY` near the top, and change `0.50f` to `0.30f` to slow the goose down for testing.
4. **Build and flash.** Click the PlatformIO upload button under the `mega` environment (left sidebar) or run `pio run -e mega -t upload`.

**Drive the goose.** Verify the speed change.

**Where to make common changes:**

- Speed / feel → constants near the top of `main.cpp`
- Add a new song or sound → place the MP3 in the correct SD card folder (`02/`, `04/`, etc.) and update the corresponding `_TRACK_COUNT` constant
- Add a controller binding → look at `handleHopperButton()` for a button-based example, or `handleSoundDpad()` for a D-pad-based one, and copy the pattern
- Add an LCD mode → increment `LCD_MODE_COUNT` and add a case in `updateLCD()`
- Change eye colors → search for `led_r =` and `led_g =` in the button handlers

## 8. TUNING GUIDE

Most of the robot's feel is controlled by constants near the top of `main.cpp`. Each section is commented with what it affects.

### Driving Speed and Stick Feel:

- `MAX_DUTY` (currently 0.50) caps motor power. Set to 1.0 for full speed (it's fast and needs runway to brake).
- `DEADZONE` (0.15) ignores tiny stick movements so the goose doesn't creep in the case of stick drift.
- `RESPONSE` (2.0) curves the stick response. 1.0 is linear, 2.0 gives more precision near center.

### Acceleration / Braking smoothness:

- `THROTTLE_RAMP_ACCEL` and `_DECEL` control forward/back smoothness.
- `STEERING_RAMP_ACCEL` and `_DECEL` control turning smoothness.
- Note: changing these affects each other slightly because of how tank drive mixing works (commented in code). `_ACCEL` and `_DECEL` constants were included to prevent abrupt braking and to ease the force that comes from that. We use this over the Spark MAX ramp control on Rev Hardware Client to have independent smoothness control over driving and steering

### Anti-tip aggression:

- `TILT_RESTING` is the resting pitch - calibrate this before tuning anything else.
- `TILT_THRESHOLD` (3°) when brake softening kicks in.
- `TILT_MAX` (7°) when full softening + catch boost begin.
- `CATCH_GAIN` (0.03) and `CATCH_MAX` (0.30) control how aggressively the goose tries to save itself.

### Death / failsafe boundaries:

- `DEATH_PITCH` (-20°) and `DEATH_PITCH_BACK` (60°) are the angles past which the goose plays a death sound and motors stop trying to anti-tip.
- `DEATH_RECOVERY_PITCH` (0°) and `DEATH_RECOVERY_BACK` (25°) define the safe zone for revival.

### Audio:

- `VOLUME_MAX` / `VOLUME_MIN` / `VOLUME_STEP` tune the volume control range and step size.
- Track count constants (`MUSIC_TRACK_COUNT` etc.) - **update these whenever you add or remove sound files to the SD Card.**

### Eyes:

- `BLINK_HOLD_BRIGHT`, `BLINK_CLOSE_TIME`, `BLINK_HOLD_DIM`, `BLINK_OPEN_TIME` - blink animation timing in milliseconds.
- `BLINK_DIM_LEVEL` - how dim the "closed eye" gets (0.0 = off, 1.0 = same as bright).
- `RAINBOW_SPEED` - how fast the rainbow cycles.

## 9. DEBUG GUIDE

*Not Driving / Only one wheel driving:*

1. **Check CAN wiring.** Check that all can wires have proper connections, or any of the power, ground, or white data wires have not disconnected from the Wago connectors in between either NEO Sport Motors or Spark MAXes. In addition, check the board with the terminating resistor and ensure that jumpers are making proper contact with the MCP2515 CAN shield screw terminals on CAN High/Low.
2. **Ensure Spark MAXes have power and ground.** Again, something might have slipped out of the Wago connectors, refer to [Spark MAX status light sheet](#)
3. **Check SPI wiring between the MCP2515 and the Mega.**
  - a. MISO -> Pin 50
  - b. MOSI -> Pin 51
  - c. SCK -> Pin 52
  - d. CS -> Pin 53
  - e. CAN INT -> Pin 2 (Unused, shouldn't affect anything)
4. **Check MCP2515 Power and Ground.** Should be connected to Mini Power Module
5. **Hit reset on the Mega.** Sometimes when driving on bumpy terrain, the robot would stop driving and we had to reset the Mega to continue motor operation. I believe this was patched out by dumping out garbage data from the IMU from impacts, but it wouldn't hurt to try.

*Controller not working:*

1. **Check batteries.** We have some Panasonic rechargeable AA batteries. Batteries might need charge or replacing.
2. **Resync Controller.** Hold the sync button at the top of the controller and ensure it pairs
3. **Verify Bluetooth Nano.** Ensure the Arduino Nano is on and has a solid light. Ensure it has shares common ground, power, and TX/RX connection (Nano D4 -> Mega 19 RX [Purple]).

*Speaker not working:*

1. **Check SD Card.** Sometimes you just forget to put it in
2. **Check wires.** Verify Mini Power Module connections and soldered wires on the board. Check Serial2 (16 TX, 17 RX) = DFPlayer Mini [Green, Yellow]. Ensure the speaker is on screw terminals.



*Anything else:*

1. **Reset the Mega / Unplug and Replug the Battery** - Also check battery voltage, ensure each cell is not close to dropping to 3 V, this is dangerously low.
2. **Look for any loose ground wires** - One ground wire disconnecting messes with the whole system
3. **Check all protoboards** - Solder may have come loose, especially on the board for the eyes. This happened once before where one of the RGB connections broke.
4. **Check Fuses** - The 5 Amp fuse may have broken from the power hub to the voltage step down converter.

```
// PIN ASSIGNMENTS (Arduino Mega)
// Serial1 (19 RX) = Controller input from Nano [Purple]
// Serial2 (16 TX, 17 RX) = DFPlayer Mini [Green, Yellow]
// Serial3 (14 TX, 15 RX) = Soundboard Nano (web server) [Green, Yellow]
// SPI (50 MISO, 51 MOSI, 52 SCK, 53 CS) = MCP2515 CAN [White, White, Purple,
Gray]
// I2C (20 SDA, 21 SCL) = 16x2 LCD with I2C adapter [Blue, Purple]
// 2  = CAN INT [Purple]
// 3  = Red LED (PWM)
// 4  = Green LED (PWM)
// 5  = Blue LED (PWM)
// 6  = DRV8871 IN2 propeller speed (PWM) [Green] (unused)
// 7  = DRV8871 IN1 propeller direction [White] (unused)
// 8  = Hopper servo [Orange]
```

## 10. KNOWN ISSUES / FUTURE WORK

These are areas where the project works but has rough edges, plus features that were scoped out or intended but never implemented. Each is a good target for future teams.

- **Hopper reliability.** The servo + 3D-printed hopper mechanism jams occasionally and sometimes doesn't fully open. A different dispense mechanism would help.
- **LCD mode 4 reduces driving responsiveness.** The 150 ms refresh rate for IMU tuning mode steals loop time. Switch off this mode before actually driving. This was useful for tuning anti-tip and can easily be guarded in code so it only applies when actively tuning.
- **Neck fragility.** The neck has snapped off the chassis several times under crash forces. A more flexible joint, a reinforced internal pipe, or an alternate mount would survive impacts better and be more stable. This would not be a problem with the shell design.
- **Shell and outer design.** The goose currently uses a plush exterior over a basic frame. A proper rigid 3D-printed shell exists but came out too large to use as-is. An acrylic top panel is also available as an alternative cover.
- **Anti-tip is P-only.** Catch boost anti-tip is purely proportional control on tilt. A full PID could give smoother saves, or enable true two-wheel self-balancing, at the cost of significant tuning complexity and CPU headroom.
- **Waveshare servo motors never arrived.** We ordered Waveshare 40 kg\*cm high-torque servos for redesigned legs that could bend at hip and knee. The delivery sat in a warehouse too long and was returned to China. The Waveshare serial bus driver board is already in our inventory and ready for the next iteration.
- **CAN vs PWM & PID Control.** We chose CAN for speed and responsiveness anticipating PID self-balancing. For the current three-wheel design, PWM motor control would have been more than sufficient and far simpler. If a future team pursues PID balancing, keep CAN, setup was the hardest part and it's already working.
- **Animatronic features.** Higher-torque servos could drive an articulated neck, head, mouth, and wings to add personality.
- **Wiring / PCB design.** Strongly consider protoshields for the Arduinos or a proper PCB designed in KiCad (free) or Altium (free for students).
- **Autonomy and sensors / ML stuff.** No obstacle avoidance. Adding ultrasonic, lidar, or ML-based perception would open up autonomous demo modes. There was an idea to have it spray people with water, but Kris said no.
- **DJ mode uses a flat 5-second timer.** Rather than detecting actual end-of-track, the intro-to-music transition uses a hardcoded delay because `DFPlayerPlayFinished` had false positives during testing. Dead air is possible. Either scrap the feature or rebuild it on top of more reliable end-of-file detection.

- **Speech interactivity.** A speech bot or LLM-driven Q&A mode would make the goose a more engaging showcase piece at recruiting events. There was discussion of getting one to work via a Raspberry Pi.

## 11. REFERENCES

External resources you'll likely need:

- **REV Hardware Client** - required for configuring the Spark MAX motor controllers (CAN ID assignments, motor inversion, current limits, ramp rates, telemetry tracking). Free download from REV. <https://docs.revrobotics.com/rev-hardware-client>
- **Spark MAX Status LED Patterns** - If anything is wrong with the Spark MAXes, refer to the status LED patterns: <https://docs.revrobotics.com/brushless/spark-max/status-led>
- **DFPlayer Mini documentation** - track number rules, command reference, and the SD card formatting requirements (FAT32, files renamed to 3-digit names). <https://wiki.dfrobot.com/dfr0299/>
- **Bluepad32 library docs** - for the Nano ESP32 controller side (button bitmasks, supported controllers, pairing process). <https://bluepad32.readthedocs.io/en/latest/>
- **Adafruit BNO055 datasheet** - orientation conventions, calibration procedure, supported modes: <https://www.adafruit.com/product/2472?srltid=AfmBOoq-nyW9HldLfYvGaFhKQsxb0AYZDtPsXDMOf4fH64oHFkZaUsdv>
- **Will Guimont's CANControl repo** – Repo used that gave us the ability to use the Arduino to control the Spark MAXes over CAN without the use of a roboRIO: <https://github.com/willGuimont/CanControl>
- **Spark MAX CAN API reference (REV docs)** - frame IDs for control modes and status frames. We've decoded duty cycle set, status\_1 (velocity/temp/voltage), and the heartbeat. Plenty more frame types are documented but unused: <https://docs.revrobotics.com/brushless/spark-max/control-interfaces>
- **Proper usage and handling of Lithium Polymer batteries** - LiPo batteries are extremely delicate and volatile and should be handled with utmost care. Always charge LiPo battery to storage charge (3.8 V per cell) after use and only fully charge when using within 24 hours to prevent damage: [http://runamok.tech/AskAaron/LiPo\\_FAQ.html](http://runamok.tech/AskAaron/LiPo_FAQ.html)
- **PID Control** - Play with this tool and learn about tuning PID values: <https://learnaboutpid.com/>

Files in this codebase to know about:

- GitHub Org - Repos live and can be forked from our public [GitHub organization](#)
- [CanControl-fork/src/main.cpp](#) - all the Mega code. >1000 lines. Constants at the top, setup at the bottom, helpers in between.
- [CanControl-fork/platformio.ini](#) - build configuration.

- [Goose\\_soundboard/soundboard\\_server.ino](#) - the code on the Bluetooth Arduino Nano which contains all the code for Xbox Controller connectivity.
- [Goose\\_controller/goose\\_controller.ino](#) - the code on the WiFi Arduino Nano which contains the html and web server functionality.

## 12. PROJECT HISTORY HIGHLIGHTS

Lessons and decisions worth knowing about.

### Design Philosophy

- **Three wheels over two.** Self-balancing was on the table the whole project. Without confidence in the Mega's loop rate and the IMU's speed for stable PID, the third caster wheel was added and the design got dramatically more robust. If you're committed to making PID balancing work end-to-end, you can drop the third wheel.
- **Underpromise to the client, overpromise to yourselves.** The team kept two sets of requirements, a conservative one for our client and a wishlist for internal goals. We had a lot of freedom in this project to add extra fun features. Be creative and use this project as a platform to explore your interests.
- **Plush exterior beat the printed shell at demos.** Audiences responded well to the stuffed-goose look than to the rigid 3D-printed shell. There was a plan to get fur and fabric over the 3D-printed shell but it never arrived in time. A proper shell would add a lot of integrity and polish.

### Procurement

- **Order everything before semester start.** Forgetting to order the Giga in Fall cascaded through the entire spring. It arrived April 8, too late to migrate everything properly. This ended up working out better for us however because of compatibility with the mega. The WaveShare servos never arrived, we should have ordered them over winter break instead of waiting.
- **Pad all procurement timelines by weeks.** Amazon orders routinely took weeks. The WaveShare servos *never arrived*. They sat in a warehouse and were returned to China.
- **Inventory the BOM the first week back from break.** Multiple parts were missing or wrong (gearboxes, buck converter). Catching it early gave time to re-order.
- **Lab Equipment.** For some reason the lab doesn't have a lot of screws, which prompted many trips to hardware stores.

## Hardware

- **Stay on Mega unless you're willing to redo CAN.** The Giga is 3.3V logic; MCP2515, DFPlayer, and LCD are all 5V. Migration requires level shifters on every signal. We tried; everything worked except the motors and boot was slow. Reverted. Will Guimont's code was also not tested for the Giga, but Cannon utilized aspects of the repo. You would have to buy a 3.3V logic version of the MCP2515.
- **NEO encoder connectors are fragile.** Once seated, be careful. We had to remake one once. Replacement heads in inventory.
- **Use Wago connectors, not PowerPole.** PowerPoles were tried and abandoned. Couldn't crimp reliably. Wagos are serviceable and forgiving.
- **120  $\Omega$  termination on CAN is required.** Without it, motors jitter visibly. There is also a ceramic capacitor over power and ground on the MCP2515. We did not do extensive testing on if it reduced noise or not, keep it to be safe.
- **REV Hardware Client must be pinned to v24.0.1.** I believe later versions broke compatibility with Will Guimont's repo.
- **Fix motor inversion in REV Hardware Client, not in code.** Burn the "Inverted" flag to flash on the affected Spark MAX. Keeps firmware clean; the inversion lives with the controller where it belongs.

## Software Architecture Decisions

- **CAN was chosen for future PID, not because the current design needs it.** PWM would have been more than enough for three wheels. If you're not pursuing PID balancing, you could simplify by going back to PWM, but the CAN setup was the hardest part and it already works, so probably just keep it.
- **The Will Guimont repo.** <https://github.com/willGuimont/CanControl> reverse-engineers the roboRIO's CAN protocol so an Arduino can drive Spark MAXes without a \$400 roboRIO.
- **Bluepad32 setup is painful but reliable once working.** Bluepad32 replaces the standard ESP32 board package. Nano ESP32 DFU upload requires a double-tap reset to enter bootloader mode (timing matters, it is actually half a second between presses). USB Mode must be set to "Debug" or serial output silently breaks.
- **Anti-tip is P-only on purpose.** Full PID is more aggressive tuning and meaningful CPU cost. P-only catches most almost-tips without needing to tune integrators.

## LiPo Safety

- **Never discharge a cell below ~3.0V.** One battery was over-discharged early and had to be disposed of. Ensure every cell is read by the charger before charging. Always check

for swelling or damage to the battery. Contact Kris Schindler immediately when a LiPo is unusable.

- **Storage charge to 3.8V/cell** between sessions. Only charge to full when using within 24 hours. *Refer to Robo-Goose user manual for LiPo battery instructions.*
- **Unplug the battery before any wiring change.** This rule is non-negotiable. Avoid frying components.
- **Battery percentage from voltage is approximate.** Accurate at idle, sags slightly under load.
- **Order LiPo Safe BatBoxes.** LiPo fires are no joke. Contact UBPD in the event of a LiPo fire. Encourage Kris to order LiPo safe storage containers for the lab.

## Workflow Lessons

- **DREAM Lab can be unreliable.** Plan around print failures and long queues. One of our team members eventually bought a personal printer for the lab, invaluable in the final stretch of this project.
- **Test drive the robot publicly before the official demo.** Test drives around campus surfaced bugs and got the team comfortable presenting. Hidden flaws (CoM shifts, neck fragility, hopper jams) only appeared with real use.
- **Add a debug guide to documentation.** Tip stolen from Robot Flock's client. Every project should have a "if it doesn't work, what's the first thing to try?" doc written down.
- **The soundboard web app was a late addition and a hit at demos.** All it took was one extra Nano. Surprisingly increased interactivity by a large amount.

## Key People

Robo-Goose initially started as a project for UB IEEE, but the project was costly and not accessible for underclassman. Thus, it was pitched as a senior design project for CSE450/453. The robot was initially conceived as a bipedal walking robot with a gyroscope mechanism used to shift center of balance. Integrating features of mechanical design and control, as well as behavior and automatic control based off environmental stimulus. The following people are both in our Discord server, they are former IEEE members who will (hopefully) be future clients for this project:

- **Thomas Mehok, Computer Engineering BS '25** - Engineer at MOOG who works on motor control systems.
- **Hailey O'Mara, Mechanical Engineering BS '25** - Currently pursuing a Masters in Animatronics.

## 13. APPENDIX

### 13.1 Body Equation

Calculating Center of Mass:

$$x_{cm} = \sum m_i x_i / \sum m_i$$

$$y_{cm} = \sum m_i y_i / \sum m_i$$

$$z_{cm} = \sum m_i z_i / \sum m_i$$

Example:

If you put a 0.5 kg battery at  $x = +/- 5\text{cm}$

And a 0.3 kg motor at  $x = -10\text{ cm}$

Then the CoM along x is:

$$x_{cm} = (0.5) \times (5) + (0.3) \times (-10) / 0.5 + 0.3$$

Mass estimation formulas:

$$m = \rho V$$

$$m = \rho A t$$

$m$  = mass (kg)

$\rho$  = material density (kg/m<sup>3</sup>)

$V$  = volume of body/component(m<sup>3</sup>)

## 13.2 Approximating battery power

Assumptions:

- Only 80% of the 10Ah battery capacity is used
- Battery voltage is approximated as the nominal 11.1V
- Each REV NEO Sport Motor is assumed to draw ~9A average current during driving
- Arduino and peripheral electronics are approximated to draw ~1A total current
- \*\*Losses from wiring, battery internal resistance, and the Spark Max efficiency are neglected.

### Running Motors at 100%:

Drive motors: ~18A Arduino Peripherals: 0.5–2A total  $10\text{Ah} \times 80\% = 8\text{Ah}$   $8\text{Ah} \div 20\text{A} = 0.4$  hours 24 Minutes each battery giving about 24 minutes of CONSTANT driving at max speed and operation

### Running motors at 35%:

Drive motors:  $18\text{A} \times 35\% = 6.3\text{A}$  Arduino Peripherals: .5-2A total  $8\text{Ah} \div 7.5\text{A} = 1.07$  hours 60 Minutes **each battery giving ~60 minutes of constant driving and operation at 35%**

*Adding self-balancing would affect this greatly. We actually saw much greater battery life during demos, because these calculations were done assuming constant driving and high current draws.*